Ocean Engineering 252 (2022) 111101

Contents lists available at ScienceDirect

Ocean Engineering

journal homepage: www.elsevier.com/locate/oceaneng

An optimal coverage path plan for an autonomous vehicle based on polygon decomposition and ant colony optimisation ☆

Karthikeyan Mayilvaganam *, Anmol Shrivastava, Prabhu Rajagopal

Centre for Non-Destructive Evaluation , Department of Mechanical Engineering, Indian Institute of Technology Madras, Chennai, India

ARTICLE INFO

Keywords:
Coverage path plan
Polygon decomposition
Path planner
Trajectory optimisation

ABSTRACT

This paper proposes a coverage path planner for the autonomous vehicle used in inspection operation. The proposed coverage path planner is developed based on a back and forth pattern (equivalent to boustrophedon pattern), producing the output path for the inspection operation with the least trajectory length and number of turns covering the entire workspace. Implementing a back and forth pattern on an arbitrary polygon (extracted from workspace boundaries) is not always viable due to the existence of concave vertices in the polygon. So, this work proposes the tree search model based on the polygon width and back and forth pattern's sweep line, which decomposes the input polygon in numerous ways. The Dijkstra algorithm solves this tree model to obtain back and forth pattern viable polygons with the least number of turns. Later, the vehicle visits these polygons one by one during the inspection operation. The order in which polygon visits are made by the vehicle affects the overall trajectory length. So, this work proposes the ant colony optimisation based trajectory length optimiser, which finds the polygon visiting order that yields minimum trajectory length. The developed coverage path planner is versatile, and it applies to any autonomous vehicle that operates in the planar region.

1. Introduction

As the world enters an era of rapid automation, autonomous vehicles have gained wide traction, replacing hazardous manual operations. Unmanned Aerial Vehicle (UAV), Autonomous Underwater Vehicle (AUV) and Autonomous Ground Vehicle (AGV) operate in the air, water and land, respectively, and are engaged for operations such as bathymetry, countermine exploration and aerial inspection. Regardless of the medium in which such vehicles operate, they require path planning algorithms guiding them in the workspace, typically achieved through offline path planners. These path planners requirements vary according to the nature of the application.

Most classes of path planners find the path between the vehicle's current position and the target position for the vehicle. These path planners generate the waypoints, enabling the vehicle to reach the target location. The well-known path planners are available, and a few of them are rapidly exploring random tree-based path planners (LaValle, 2006), particle swarm optimisation based path planners (Chai et al., 2021) and neural network-based path planners (Yu et al., 2020). Cooperative vehicle path planner is another important domain where researchers actively developing path planners for the multiple vehicles cooperatively doing the task (Tsourdos et al., 2010). As far as an autonomous vehicle used in the inspection operation such as bathymetry (underwater inspection) and aerial inspections are concerned, there is a need for a unique path plan which is called Coverage Path Plan (CPP). The CPP ensures that vehicles explore the whole region under survey, and the requirement for CPP is different from all other path planners. The work reported in the paper focuses on developing the algorithm for the CPP.

The CPP depends on the polygon formed by the survey region's boundaries and defines the set of points inside the polygon in a row and column fashion. Later, these points are connected in a specific manner to form the path that passes through every defined point. This path directs the vehicle to accomplish the designated task. Different patterns such as scan or Back and Forth (BF), Hilbert and localisation algorithms with a mobile anchor are known in the literature for joining these defined points (Artemenko et al., 2016). Artemenko et al. (2016) explain that the number of turns and trajectory length significantly affects the vehicle's energy requirement. An efficient CPP should have the least number of turns and trajectory length. Reducing the number of turns reduces the energy requirement, as also explained by Zhu et al. (2019) and Dogru and Marques (2015). Also, more turns increase the

☆ This document is the results of the research project funded by the National Technology Centre for Ports, Waterways & Coasts, Indian Institute of Technology Madras.

* Corresponding author.
E-mail addresses: karthikeyan311994@gmail.com (K. Mayilvaganam), anmolvj15@gmail.com (A. Shrivastava), prajagopal@iitm.ac.in (P. Rajagopal).

https://doi.org/10.1016/j.oceaneng.2022.111101
Received 17 October 2021; Received in revised form 26 January 2022; Accepted 12 March 2022
Available online 8 April 2022

operation time of the vehicle (Dogru and Marques, 2015). As discussed many patterns available for the CPP; each has its own advantages and disadvantages, which are explained well in Artemenko et al. (2016) and Cabreira et al. (2019). The well-known BF pattern is adopted in this work because the applicability of this pattern is vast, and it is also recommended for the large operational area (Cabreira et al., 2019).

In the BF based CPP, the points are defined inside the workspace polygon in a row and column manner (at a regular distance interval) at any arbitrary orientation. The points are then connected in a row by row or column by column, resembling the boustrophedon pattern. These row or column shifting connections signify the turns that need to be taken by the vehicle. The BF pattern orientation is constrained by the polygon type concave and convex, and this polygon type is classified based on the interior angle (Cabreira et al., 2019; Huang, 2001; Jiao et al., 2010; Torres et al., 2016). For a convex polygon, all the interior angles are less than π, whereas, for the concave polygon, at least one of the internal angles is greater than π. The BF configuration can be oriented at any arbitrary direction in a convex polygon but constrained by concave vertex (vertex interior angle greater than π) in a concave polygon.

The specific orientation of BF based CPP implementation on the polygon leads to the least number of turns along the polygon width for the convex polygon (Torres et al., 2016). The polygon width is calculated using two parallel line support, which forms an antipodal pair (Shamos, 1975; Li et al., 2011; Pirzadeh, 1999; Gomez et al., 2017). The orientation for the BF pattern is impossible for some concave polygons and feasible for other concave polygons, and it depends on the concave vertices. If the BF implementation is not viable in a particular concave polygon, it is decomposed into many sub-polygons recursively until the configurations are feasible in all sub-polygons (Cabreira et al., 2019; Choset, 2000; Jiang et al., 2018). Once the input polygon is made viable for the BF pattern implementation through recursive decomposition, the optimal BF based CPP is applied on each polygon separately.

The autonomous vehicle sequentially visits the decomposed sub-polygons to complete the task. The order in which the polygons are visited influences the overall trajectory length (Jiao et al., 2010). The overall trajectory length is the summation of each polygon's CPP trajectory length and each transition distance (the distance travelled by vehicle from one to another polygon in the same workspace for operation). The transition length between the polygons depends on the sequence in which the robotic vehicle visits the polygons, and this needs to be optimised.

It is also evident from the literature (Jiang et al., 2018; Shi and Zhou, 2020; Cabreira et al., 2018) that one of the major applications for autonomous vehicles is inspection, such as bathymetry and aerial photogrammetry. This inspection operation required a unique offline path plan that is CPP, which motivates the development of a special path planner for autonomous vehicles in this work. This paper aims to develop an algorithm for generating CPP on any polygon that represents a workspace while optimising the number of turns and trajectory length. This paper provides the mathematical formulations, and the algorithm depends on Dijkstra Algorithm (DA) (LaValle, 2006) and Ant Colony Optimisation (ACO) (Li et al., 2008; Ariyasingha and Fernando, 2015) built over these formulations for obtaining the mentioned objective. Section 2 presents the background definitions which are vital for the proposed CPP method. Section 3 presents proposed CPP algorithm in detail. Section 4 explains the implementation of the proposed method, along with results and discussion. Section 5 concludes this work.

2. Background definitions

In this Section, the polygons obtained from the simple workspace boundaries are used to explain the various definitions and their mathematical relationships that are essential for building the proposed algorithm. The formulations for implementing the BF based CPP in an arbitrary polygon is explained in Section 2.1.

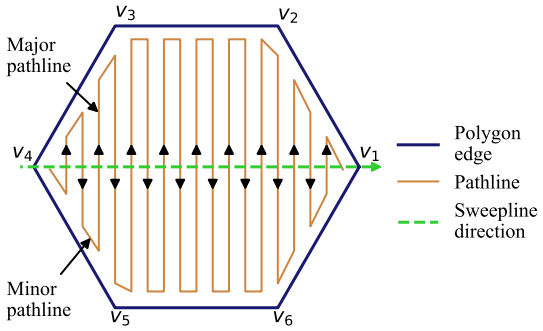


Fig. 1. Schematic diagram showing the BF based CPP over a convex polygon.

2.1. Back and forth based coverage path plan

The BF based CPP implementation on the convex and concave polygon varies and is explained in Sections 2.1.1 and 2.1.2.

2.1.1. Back and forth in a convex polygon

The BF based CPP for an arbitrary convex polygon is shown in Fig. 1, whose vertices are v_1, v_2, v_3, v_4, v_5 and v_6. The path generated is classified into major and minor path lines. The vehicle performs its intended operation, such as inspection on major path lines. The minor path line is the one used to shift from one major path line to another. Fig. 1 indicates that the BF pattern gets filled in the polygon in the sweep line's direction and is normal to each major path line. It is desirable to have the path with the least number of turns and minor path lines, and it can be obtained by finding the optimal sweep line direction.

Best sweep line direction

Any arbitrary sweep line orientation for the BF pattern covers the whole convex polygon. Still, specific sweep line directions have the least number of turns. This better sweep direction can be found by finding the polygon spans and determining the polygon width from the spans (Torres et al., 2016; Shamos, 1975). The spans are calculated using the parallel line of support and the antipodal pair concept.

Definition 1. A line L is said to be a line of support for the polygon P if it is tangent to one or two vertices such that the entire polygon lies on one side of the line (Pirzadeh, 1999).

Definition 2. A pair of vertices in the polygon P is said to be antipodal pair if both the vertices in the polygon admit the line of support such that the lines are parallel (Pirzadeh, 1999).

Definition 3. A span (D) of a polygon is defined as the perpendicular distance between the two vertices that admit a parallel support line (Li et al., 2011). A single polygon can have many spans.

Definition 4. A polygon width is the least distance span (D) of the polygon (Jiao et al., 2010).

Based on Definitions 1–3, they are three types of spans available, which are Vertex–Vertex (V–V) style, Vertex–Edge (V–E) style and Edge–Edge (E–E) style are shown in Figs. 2 (a–c).

Suppose the sweep line orientation for the BF based CPP aligns along any span of the polygon. In that case, the relation between the number of turns (n_{turns}), length of the respective span (L_{span}) and distance between two subsequent major path lines (L_{mpl}) is expressed as per Eq. (1). It is inferred from Eq. (1) that the sweep line orient along the polygon width leads to the least number of turns.

$$n_{turns} = \frac{L_{span}}{L_{mpl}} - 1 \tag{1}$$

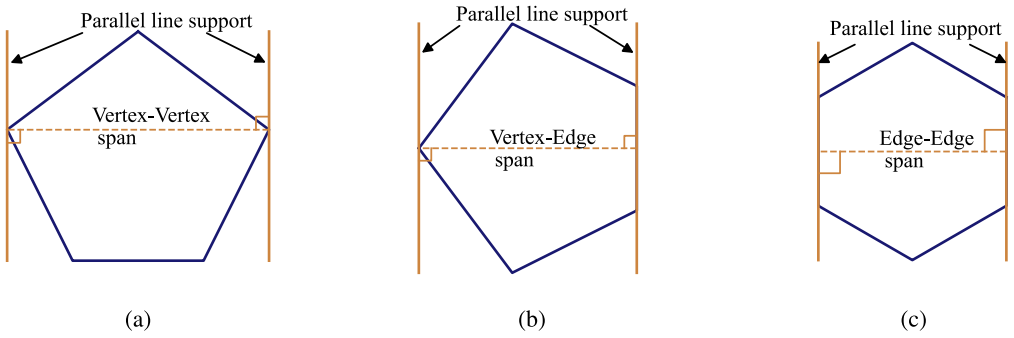


Fig. 2. Schematic diagram showing (a) Vertex–Vertex, (b) Vertex–Edge and (c) Edge–Edge style span for a convex polygon.

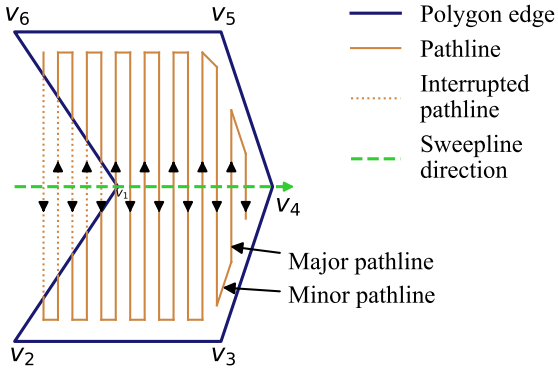


Fig. 3. Schematic diagram showing the BF based CPP over an arbitrary concave polygon.

2.1.2. Back and forth in a concave polygon

The BF based CPP for the concave polygon cannot be implemented in any arbitrary sweep line direction, as shown in Fig. 3. The path lines are interrupted by the polygon edges in the concave polygon if a particular sweep line direction is chosen. It is necessary to determine the constrained conditions for the feasible sweep line direction in a concave polygon.

Feasible sweep line orientation

The viable sweep line direction for the concave polygon can be found using the concave vertex visible region (Li et al., 2011; Jiang et al., 2018). The visible region for the arbitrary concave polygon with one concave vertex is shown in Fig. 4(a). The concave vertex is one whose interior angle is greater than π. For any concave vertex, the respective two edges forming the vertex are extended inside the polygon area. In Fig. 4(a), the extended lines for concave vertex v_1 are $v_6 - {v_1}^1$ and $v_2 - {v_1}^1$. The BF based CPP is feasible only when the major path line orient between $v_6 - {v_1}^1$ and $v_2 - {v_1}^1$ covers the concave polygon shown in Fig. 4(b), and the respective sweep line direction is normal to the major path line. In BF based CPP, the neighbouring major path lines are parallel and opposite.

Suppose a concave polygon has more than one concave vertex. In that case, the feasible major path line for the BF based CPP must orient along each concave vertex visible region or the common visible region computed from each concave vertex visible region. The visible area and common visible region for the concave polygon with two concave vertices are shown in Figs. 5(a–b). If the common visible region is not present, then the polygon must be decomposed so that each decomposed polygon has the feasibility of implementing the BF based CPP.

Best sweep line direction for viable concave polygon

Many major path line orientations for BF based CPP, which lie in the common visible region, are possible for the feasible concave polygon. But a particular orientation has a fewer number of turns available. This work uses the convex hull based span to find the major path line orientation with the least number of turns. The convex hull is the minimum area convex polygon, inscribing the whole concave polygon inside (Torres et al., 2016). Fig. 6(a) represents the concave polygon with its convex hull. It is mentioned in Section 2.1.1 that the polygon width is one of the V–E style spans. So, the convex hull is considered as a convex polygon, and respective V–E style spans are calculated as shown in Fig. 6(b).

Each V–E span's normal orientation is tested for alignment along the visible region of each concave vertex or common visible region. The minimum length V–E span's normal that satisfies all visible regions of the concave vertices or common visible regions chosen as major path line orientation for the BF based CPP. For some instances, none of the V–E style spans lies in the visible regions. However, some V–V span is feasible. In this work, the polygon is still treated as an infeasible concave polygon because width lies in the V–E span, which provides the least number of turns when BF based CPP is applied over the polygon. The BF based CPP non-viable polygon is decomposed into multiple polygons recursively until anyone V–E span is viable for BF based CPP implementation, and the decomposition is explained in Section 2.2.

2.2. Concave polygon decomposition

The concave polygon for which BF based CPP implementation not viable is decomposed. The decomposition procedure can be done based on the visible region concept given by Jiang et al. (2018). The decomposition procedure consists of three steps: identifying the concave vertices, finding the alternate points for each concave vertex, and decomposing the polygon by forming an edge between the concave vertex and its alternate point.

2.2.1. Identification of concave vertices

The edge vector is generated on the polygon edge in the counterclockwise direction to identify the polygon's concave vertices. The portion of the simple polygon with edge vectors are shown in Fig. 7.

The vertex v_i is said to be a convex vertex if $|\overline{v_{i-1}v_i} \times \overline{v_iv_{i+1}}|$ greater than zero and concave vertex if $|\overline{v_{i-1}v_i} \times \overline{v_iv_{i+1}}|$ less than zero. The condition $|\overline{v_{i-1}v_i} \times \overline{v_iv_{i+1}}|$ equals zero implies that the two vectors are collinear, and such a case is not possible in this work because edges of the polygon are chosen such that they are not collinear. Each polygon vertex is evaluated for vertex type.

2.2.2. Finding alternate points for the concave vertex

The visible region for the concave vertex in the polygon is shown in Fig. 8. The vertices that lie inside the visible region are known as alternate points. In Fig. 8, v_{j-1}, v_j and v_{j+1} are the vertices present in the visible region of concave vertex v_i. The alternate points for each concave vertex in the polygon are found similarly.

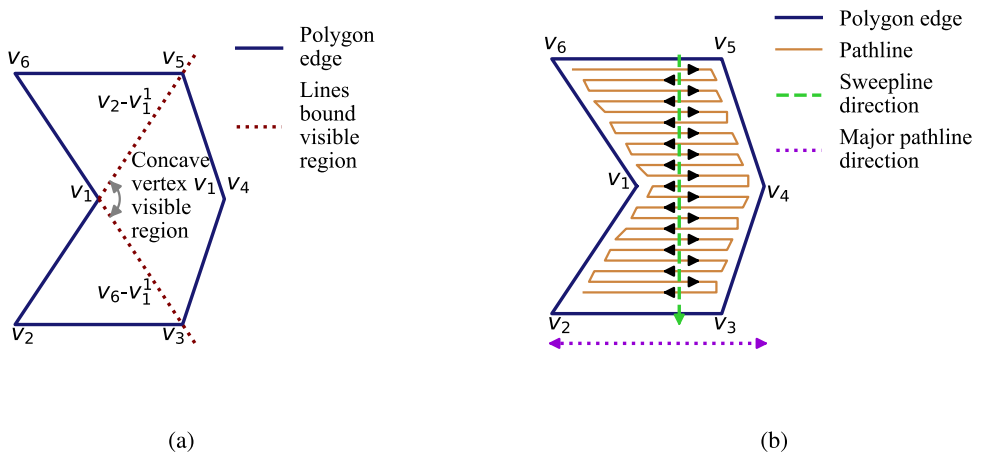


Fig. 4. Schematic diagram showing the (a) visible region and (b) feasible BF based CPP for an arbitrary concave polygon.

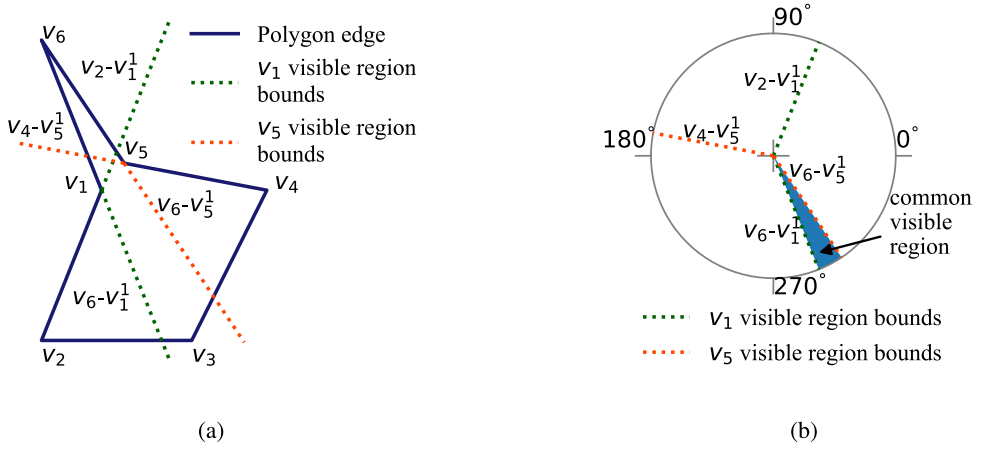


Fig. 5. Schematic diagram showing the (a) visible region boundaries and (b) common visible region for an arbitrary concave polygon with two concave vertices.

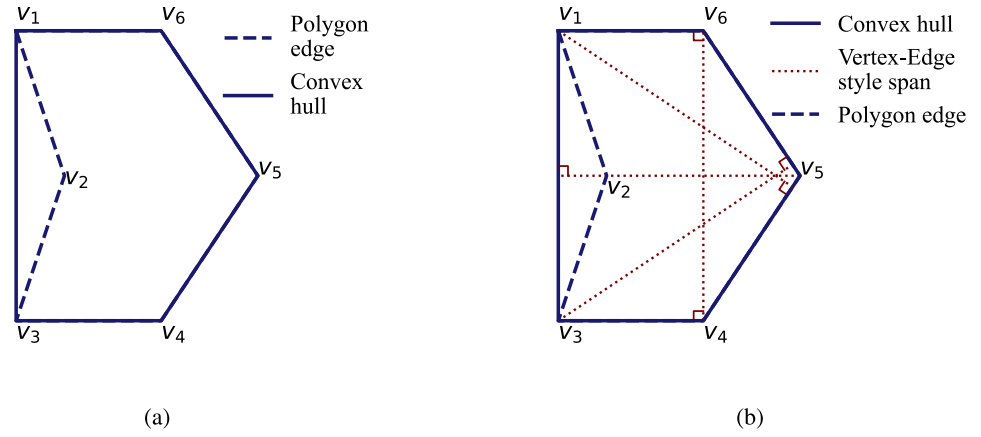


Fig. 6. Schematic diagram showing the (a) convex hull and (b) Vertex–Edge span for the concave polygon.

2.2.3. Decomposition of the concave polygon

The decomposition aims to eliminate the concave vertices and generate multiple decomposed polygons on which BF based CPP implementation is viable. Joining the concave vertex with its anyone alternate point forms the new edge and decomposes the single polygon into two polygons. Depending on the nature (concave or convex) and availability of the alternate point, three types of decompositions are viable and are illustrated using the arbitrary polygon shown in Fig. 9(a).

1. **Type I:** Decompose the polygon by joining the concave vertex with its concave alternate point, and as shown in Fig. 9(b), the concave vertex v_2 is connected with the concave alternate point v_5 to form a new edge for decomposition.
2. **Type II:** Decompose the polygon by forming an edge between the concave vertex and its convex alternate point. Fig. 9(c) shows concave vertex v_5 connected with their convex alternate point v_1 to form a new edge for decomposition.

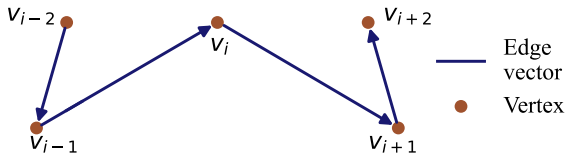


Fig. 7. Schematic representation of the edge vector's forming the portion of the concave polygon.

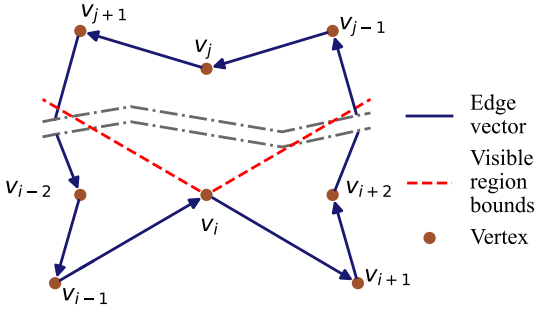


Fig. 8. Schematic diagram showing the visible region and vertices present in the visible region for the arbitrary concave polygon.

3. **Type III:** Decompose the polygon by forming an edge using an angle bisector of the concave vertex's visible region and is shown in Fig. 9(d), the angle bisector for the concave vertex v_5 visible region intersects the polygon edge, which generates a new vertex v_8. The edge formed between v_5 and v_8 decomposes the polygon. One important feature of type III decomposition is that it is always viable, which is not the case in type I and II decompositions, and this feature ensures the proposed algorithm can decompose any arbitrary concave polygon into a set of convex polygons.

Decomposition type preferences

This work applies decomposition operation recursively on the polygon until resultant polygons from the operations become convex. At any instance, multiple choices are available for decomposing the polygon based on type I, II and III. Among decomposition types, type I is always preferred over others because a single operation can eliminate two concave vertices if both are alternate points to one another. The new vertex is generated in type III, so type II is preferred over type III. This preference is considered in the proposed CPP algorithm.

3. Proposed coverage path plan method

The polygon is extracted from the workspace map is provided as the input to the proposed algorithm. The developed algorithm has multiple steps listed below as per the order and is explained in Sections 3.1–3.4.

1. **Decomposition operation:** The input polygon is decomposed into multiple convex polygons based on the sweep line orientation formulation, which is solved by DA such that the polygons width summation is least.
2. **Unification operation:** The output polygons from decomposition operation are unified based on the visible region and sweep line formulation, which is solved by DA such that the number of polygons is reduced and polygon width is maintained reasonably.
3. **Waypoint generation:** Generate the waypoints on the polygons obtained from the unification operation.
4. **Trajectory length optimisation:** It is discussed in Section 1 the transition distances affect overall trajectory length. These polygon transition distances depend on the order in which the vehicle visits each polygon for the operation, and these order of visits are optimised to reduce overall trajectory length in this step.

The proposed CPP is illustrated using the polygon section extracted from Chennai's Kasimedu fishing harbour, as shown in Fig. 10 and the implementation of CPP for the whole fishing harbour is explained in Section 4.

3.1. Dijkstra based polygon decomposition

The convex type input polygon is not decomposed into sub-polygon because BF based CPP is feasible in any arbitrary sweep line orientation, as discussed in Section 2.1.1. The decomposition procedure is applied only for the concave polygon. This step aims to divide the single concave polygon into multiple sub-convex polygons such that the summation value of each sub-convex polygon's width is the least. So, when the BF based CPP is implemented on each decomposed polygon leads to the summation of each polygon's number of turns least. This objective can be achieved by solving the mathematical formulation explained in Section 3.1.1 by the DA.

3.1.1. Decomposition formulation

The input polygon has multiple concave vertices, and each concave vertex has multiple alternate points. So, when a single decomposition operation is planned on the initial input polygon, several ways a decomposition (Type I, II and III) can be done and is shown in Fig. 11 for an arbitrary polygon. It is also inferred from Fig. 11 that recursively applying various decomposition operations leads to many groups of decomposed sub-convex polygons at the end.

Fig. 11 resembles the tree, which has all the possible decomposed sub-polygons obtained due to recursive decomposition. This tree structure has a specific group of decomposed sub-polygons at the end whose respective widths summation is the least. So, the search problem is formulated on the tree structure to find the group of decomposed sub-polygons whose widths summation is least.

Polygon definition and nomenclature

It is crucial to mathematically express the group of polygons (decomposed polygons) obtained after every decomposition operation as a set and the polygon vertices. The polygon set and vertices definitions are useful in defining the general tree structure for any polygon that undergoes recursive decomposition. The nomenclature to be followed for the polygon set and each polygon's vertices are illustrated for the specific group of polygons taken from Fig. 11 is shown in Fig. 12.

The group of polygons is denoted by polygon set S as per Eq. (2) in this work.

$$S = \{\ P^{pn} \mid pn \in \mathbb{W},\ pn < N_p\ \} \tag{2}$$

Where N_p is the number of polygons in the respective group of polygons, P is the notation that represents polygon, pn is the local polygon identification number used to differentiate the polygons in the same polygon set, and $\mathbb{W}$ is the whole number set. The polygon set for the group of polygons shown in Fig. 11 is given in Eq. (3).

$$S = \{\ P^0\ ,\ P^1\ ,\ P^2\ \} \tag{3}$$

Each polygon is represented by its vertices in the anticlockwise direction as per Eq. (4) in this work.

$$\mathbf{V}(\ P^{pn}\ ) = \left[\ p_{ij}\ \right]_{L\times 2}\ ,\ 1 \le i \le L\ ,\ 1 \le j \le 2, \tag{4}$$

Where L is the number of vertices in the polygon whose local polygon identification number in the set is denoted as pn, $p_{i1} \in \mathbb{R}$ implies abscissa and $p_{i2} \in \mathbb{R}$ implies ordinate for the vertex i in the polygon. The vertices for the polygon P^1 in Fig. 11 are given in Eq. (5). The first row coordinates in Eq. (5) can be any vertices,

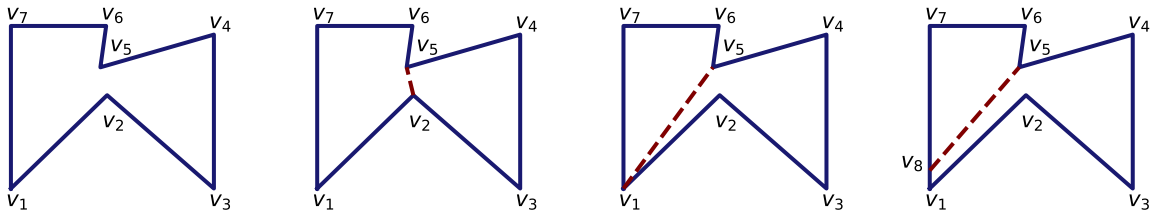


Fig. 9. Schematic diagram showing the (a) arbitrary concave polygon, and its (b) type I, (c) type II and (d) type III decomposition operation.

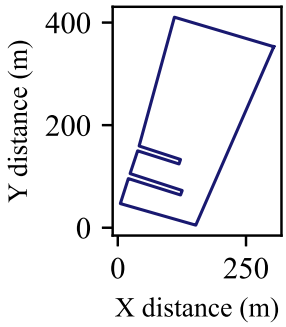


Fig. 10. Graph showing the concave polygon section extracted from Chennai's Kasimedu fishing harbour.

but the succeeding rows must be the coordinates of vertices in the anticlockwise direction.

$$\mathbf{V}(\ P^1\ ) = \begin{bmatrix} 0 & 0 \\ 1 & 0.97 \\ 0.93 & 1.26 \end{bmatrix} \tag{5}$$

Tree definition and nomenclature:

The general tree structure for any polygon that undergoes recursive decomposition is shown in Fig. 13.

The group of polygons obtained after various decomposition is represented as a node in the general tree. The node in the general tree is identified by a variable id, as shown in Fig. 13. The input polygon is given to the initial node whose id is 0 in the tree at the zero-level state. The input given to node 0 can also be a group of polygons that lies closer to one another. The group of polygons at any node in the tree is defined as a set, and the defined respective set is expressed in Eq. (6), which is the redefinition of Eq. (2) for the tree.

$$S_{id} = \{\ P_{id}^{pn}\ \mid\ pn \in \mathbb{W}\ ,\ pn < N_p^{id}\ \} \tag{6}$$

Where S_{id} is the polygon set for the node whose identification variable is denoted as id and N_p^{id} is the number of the polygon in the respective node's polygon set. The vertices for any polygon in the polygon set are defined as per Eq. (7) which is the redefinition of Eq. (4) for the tree.

$$\mathbf{V}(\ P_{id}^{pn}\ ) = \left[\ p_{ij}\ \right]_{L\times 2}\ ,\ 1 \le i \le L\ ,\ 1 \le j \le 2, \tag{7}$$

On node 0, many decomposition operations are possible. These operations generate many nodes at the first-level state, as shown in Fig. 11, and each of these nodes has a polygon set. The node that undergoes decomposition operation is called the parent node and the resultant nodes obtained from the operation are known as the child nodes. The various decomposition operations generate many child nodes and are denoted as a branch in the tree connecting the child to the respective parent. Whenever the branches evolve from any node in the tree are assigned a number starting from zero. This number is also called a branch number. The child nodes obtained at any level are identified by variable id is a concatenation of the child node's parent identification variable id and the respective branch number to reach the node from the parent by '-', and it can also be inferred from Fig. 13. Then, the new child nodes are considered the new parent. The decomposition operation is recursively done on these decomposed polygons until all the polygon in the child nodes becomes convex, as shown in Fig. 11 and generally represented in Fig. 13.

Every decomposition operation represented in the tree divides the single polygon present in the parent polygon set into two polygons. This decomposition operation can be expressed as simple matrix multiplication. The polygon's type I and II decomposition operation is expressed mathematically in Eqs. (8)–(9).

$$\begin{bmatrix} \mathbf{V}(\ P_{cid}^{pn}\ ) \\ \\ \mathbf{V}(\ P_{cid}^{N_p^{pid}}\ ) \end{bmatrix} = \mathbf{D_M}\ \mathbf{V}(\ P_{pid}^{pn}\ ) \tag{8}$$

$$\mathbf{D_M} = \begin{bmatrix} \mathbf{0}_{1\times(l1-1)} & 1 & \mathbf{0}_{1\times(l2-l1-1)} & 0 & \mathbf{0}_{1\times(L-l2)} \\ \mathbf{0}_{(l2-l1-1)\times(l1-1)} & \mathbf{0}_{(l2-l1-1)\times 1} & \mathbf{I}_{(l2-l1-1)\times(l2-l1-1)} & \mathbf{0}_{(l2-l1-1)\times 1} & \mathbf{0}_{(l2-l1-1)\times(L-l2)} \\ \mathbf{0}_{1\times(l1-1)} & 0 & \mathbf{0}_{1\times(l2-l1-1)} & 1 & \mathbf{0}_{1\times(L-l2)} \\ \mathbf{I}_{(l1-1)\times(l1-1)} & \mathbf{0}_{(l1-1)\times 1} & \mathbf{0}_{(l1-1)\times(l2-l1-1)} & \mathbf{0}_{(l1-1)\times 1} & \mathbf{0}_{(l1-1)\times(L-l2)} \\ \mathbf{0}_{1\times(l1-1)} & 1 & \mathbf{0}_{1\times(l2-l1-1)} & 0 & \mathbf{0}_{1\times(L-l2)} \\ \mathbf{0}_{1\times(l1-1)} & 0 & \mathbf{0}_{1\times(l2-l1-1)} & 1 & \mathbf{0}_{1\times(L-l2)} \\ \mathbf{0}_{(L-l2)\times(l1-1)} & \mathbf{0}_{(L-l2)\times 1} & \mathbf{0}_{(L-l2)\times(l2-l1-1)} & \mathbf{0}_{(L-l2)\times 1} & \mathbf{I}_{(L-l2)\times(L-l2)} \end{bmatrix} \tag{9}$$

Where $\mathbf{D_M} \in \mathbb{R}^{(L+2)\times L}$ is the decomposition operation matrix, pid is the notation that represents the variable id of the parent, P_{pid}^{pn} is the polygon present in the parent polygon set whose local polygon identification number is denoted by notation pn and its vertices are denoted by $\mathbf{V}(\ P_{pid}^{pn}\ ) \in \mathbb{R}^{L\times 2}$ defined as per Eq. (7), and L denotes the number of vertices in the polygon P_{pid}^{pn}. As discussed in Section 2.2.3, the type I and II decomposition operation is done by forming an edge between the concave vertex and its alternate points. This respective concave vertex and its alternate points coordinates existing row numbers in polygon vertices definition $\mathbf{V}(\ P_{pid}^{pn}\ )$ are found. Among these two row numbers least is denoted by $l1$ and the other by $l2$. cid is the notation that represents the variable id for the child node evolved from a parent node. N_p^{pid} is the number of the polygon in the parent polygon set. $P_{cid}^{N_p^{pid}}$ and P_{cid}^{pn} are the polygons obtained as a result of decomposition, and the respective vertices are denoted by $\mathbf{V}(\ P_{cid}^{N_p^{pid}}\ ) \in \mathbb{R}^{(L-l2+l1+1)\times 2}$ and $\mathbf{V}(\ P_{cid}^{pn}\ ) \in \mathbb{R}^{(l2-l1+1)\times 2}$.

For type III decomposition, the new vertex coordinate obtained due to the angular bisector intersects with the polygon edge is calculated as explained in Section 2.2.3. The new vertex coordinate is added to the existing vertices in $\mathbf{V}(\ P_{pid}^{pn}\ )$ such that between its neighbour vertices according to the anticlockwise vertices arrangement. Then, the procedure similar to the type I and type II decomposition operation is followed for finding the decomposed polygon vertices.

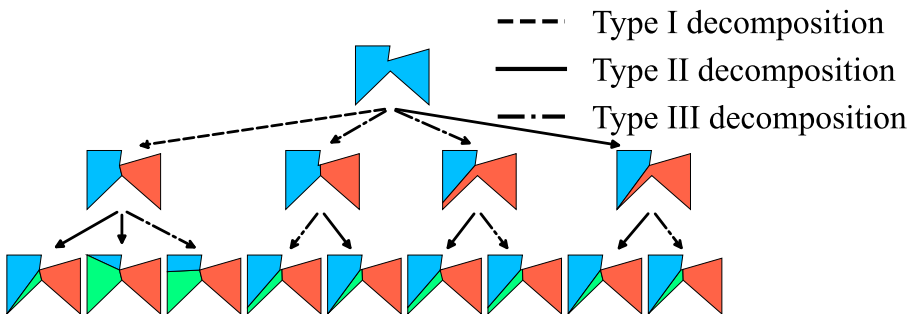


Fig. 11. Schematic diagram showing the recursive application of decomposition operation on an arbitrary polygon leads to the various decomposed convex polygons.

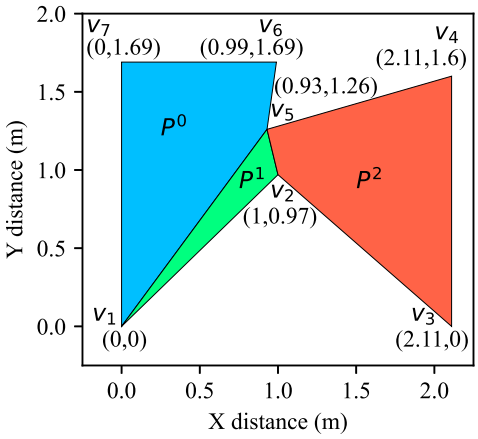


Fig. 12. Graphical representation showing the group of polygons.

After the decomposition operation, the polygon set for the child node is derived. The polygon set contains the polygons which are undecomposed in the parent polygon set and two polygons obtained due to the decomposition of the single polygon in the parent. This child node polygon set is defined as per Eqs. (10)–(12).

$$S_{cid} = S_{ud} \cup \{ P_{cid}^{pn} , P_{cid}^{N_p^{pid}} \} \tag{10}$$

$$S_{ud} = \{ P_{cid}^{k} \mid k \in \mathbb{W} , k < N_p^{pid}, k \neq pn \} \tag{11}$$

$$P_{cid}^{k} = P_{pid}^{k} \text{ such that } k \in \mathbb{W} , k < N_p^{pid} , k \neq pn \tag{12}$$

Where S_{ud} is the polygon set obtained from the polygon present in the parent node that is not decomposed, which is expressed in the Eqs. (11)–(12).

Tree node count analysis

The number of nodes in the tree shown in Fig. 13 grows abruptly if the number of concave vertices in the initial input polygon set is high. In this Section, the growth of the tree's node with respect to the number of concave vertices at the polygon set available at node 0 are analysed. The least number of nodes in the tree is obtained only if each concave vertices available at node zero has only one option for decomposition. This scenario is possible only if no alternate points are available for decomposition at any concave vertices, which leads to type III decomposition is the only viable option. It can also be understood that any other decomposition types viable with type III decomposition, leading to more options for decomposition and producing even more nodes at the next level. So, in this Section, assuming type III decomposition is the only viable decomposition at any concave vertices even after propagation in the tree due to recursive decomposition helpful in evaluating the least number of nodes possible for the tree. The least number of nodes possible (N^{least}) in the tree given number of concave vertices (N_c) available in the initial polygon set in the tree is calculated as per Eq. (13).

$$N^{least} = \sum_{i=0}^{N_c} \frac{N_c!}{(N_c - i)!} \tag{13}$$

The least number of nodes required in the tree with respect to the initial input polygon set's concave points as per Eq. (13) is shown in Fig. 14. It is evident from Fig. 14, the number of nodes requirement grows abruptly with the number of concave vertices in the input polygon set. One strategy to solve the problem with high concave vertices in the input polygon is to divide the polygon into sub-polygons such that the number of nodes requirements in the tree are less. Another strategy is to consider three decomposition types separately while building a decomposition tree; first, decompose recursively the input polygon set using type I decomposition until options for type I decomposition becomes void. Later type II decomposition and followed by type III decomposition. This order of decomposition is chosen as per decomposition preferences discussed in Section 2.2.3, and this strategy also reduces the node requirement. As far as maximum nodes requirement for the tree is concerned, it is not simple to evaluate because arbitrary polygon has concave vertices, and each concave vertex arbitrarily has many viable alternate points for decomposition. This Section aims to establish maximum node requirement for polygon set having a certain number of concave vertices is high, so it is better to subdivide polygon into many polygons to reduce node requirement, and also order of decomposition as per decomposition preferences further reduce node requirement.

Decomposition cost and objective function definition

The objective is to find the sequence of decomposition operations or the nodes in the tree that leads to the final polygon set containing the polygons whose nature is convex and their widths summation are least. The cost function has to be defined for each decomposition operation as shown in Fig. 13 as C_{0-0}, C_{0-1}, C_{0-2}, C_{0-1-0} and C_{0-1-1}. The general cost function for any node generated from the parent is denoted as C_{id}. The single polygon in the node's parent polygon set is decomposed into two polygons for any node. These two polygons are used to compute the respective decomposition operation cost. The cost value for any decomposition operation is expressed by Eq. (14).

$$C_{cid} = w(P_{cid}^{I}) + w(P_{cid}^{J}) - w(P_{pid}^{K}) \tag{14}$$

Where $w(P_{cid}^{I})$ and $w(P_{cid}^{J})$ are the widths of polygon's convex hull whose local polygon identification numbers are denoted by I and J present in the polygon set at the node whose identification variable id is denoted by cid. $w(P_{pid}^{K})$ is the width of parent polygon's whose node identification and local polygon identification are denoted as pid and K. The width calculation procedure for the convex hull is explained in Section 2.1. Suppose the cost value for the decomposition operation is less. In that case, it indicates that a significant amount of turns is reduced when BF based CPP is implemented on parent polygon.

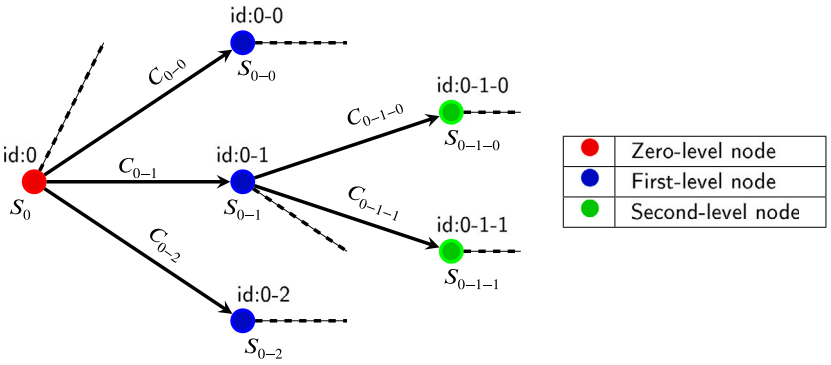


Fig. 13. Schematic diagram showing the tree represents the general input polygon set that undergoes recursive decomposition.

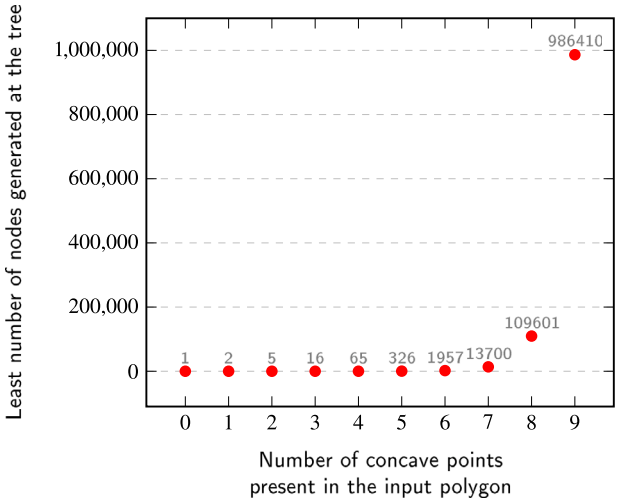


Fig. 14. Graph showing the minimum number of nodes required in the recursive decomposition tree with respect to number of concave vertices at initial node.

The DA can solve the tree with a defined cost function based on the decomposition's objective function. The decomposition's objective function for any node is expressed by Eq. (15).

$$O_{id} = C_p + C_{id} \tag{15}$$

Where O_{id} is the decomposition's objective value for the node whose identification is id and C_p is the preceding cost, which is the summation of the respective decomposition operations costs to reach the current node's parent from node 0. The Dijkstra algorithm solves this objective function for minimisation to attain the least number of turns.

3.1.2. Solving the decomposition formulation

The steps to solve the decomposed formulations to find the polygon set whose polygons width summation are least using the DA are described in this Section. Based on the preferences of decomposition type discussed in Section 2.2.3, the solving methodology first solves a tree built only by type I decomposition for a node whose polygon set has polygons whose convex hull widths summation least. Then, this polygon set is again considered an input polygon, and a tree built only with type II decomposition is solved for a node whose polygon set has polygons whose convex hull widths summation least. Subsequently, a similar procedure is followed for solving a tree built only with type III decomposition. The procedure for decomposition operation is explained below, and the respective flow chart is given in Fig. 15.

1. **Initialisation:** The polygon is extracted from the workspace. A polygon set $S_{inp} = \{ P^k \mid k \in \mathbb{W}, k < N_p^{inp} \}$ that holds the extracted polygon is provided as an input to the algorithm. The N_p^{inp} is the number of polygons present in the input polygon set. Then defined the variable $current\_id$ and $decomposition\_type$. The $current\_id$ is initialised to the variable id of the node present at zero-level, which is 0. The $decomposition\_type$ is initialised to the 'type_I' as per the decomposition preference discussed in Section 2.2.3. The polygon set for the $current\_id$, $S_{current\_id}$ is defined and initialised to the input polygon set S_{inp} because at node $current\_id$ id 0 is saved. The empty list named $node\_list$ is defined. Declare the variable preceding cost C_p and initialised to zero.
2. **Identification of concave vertices:** The vertices of all polygon in the $S_{current\_id}$ are tested for vertex nature, whether concave or convex, as discussed in Section 2.2.1.
3. **visible region calculation:** The visible region is calculated for all the concave vertices found in all polygons in the $S_{current\_id}$ at step 2.
4. **Alternate points and angle bisector calculation:** If the $decomposition\_type$ is 'type_I' or 'type_II', then the alternate points are calculated for all concave vertices in each polygon available at $S_{current\_id}$ as explained in Section 2.2.3. Suppose the $decomposition\_type$ is 'type_III', then for each concave vertex, the respective visible region angle bisector is drawn, which intersects the respective polygon edge in the visible region creates the new vertex is considered as an alternate point as briefed in Section 2.2.3. By following this procedure, all alternate points are calculated.
5. **Identification of viable decomposition:** Based on the concave vertices and alternate points found in steps 3 and 4 for all the polygon in $S_{current\_id}$, the viable decomposition operations for the current $decomposition\_type$ are calculated. If the $decomposition\_type$ is 'type_I' and the respective decomposition viability is null, then go to step 10. If the $decomposition\_type$ is 'type_II', and the decomposition viability is null, go to step 11. If the $decomposition\_type$ is 'type_III', and the respective decomposition viability is null, then go to step 12.
6. **Allocating the identification to the child nodes:** The possible decomposition found in step 5 is considered a branch in the tree from a node whose variable id same as $current\_id$. The branch number is allocated starting from zero. Considering $current\_id$ as a parent and the respective child nodes identification variable id are computed, which is a concatenation of the parent's id and the respective viable operation branch number by '-'.
7. **Finding the polygon set for the child nodes:** For each child node obtained in step 6, the respective decomposed polygon are calculated as per Eqs. (8)–(9). Then, the polygon set for each child node is calculated as per Eqs. (10)–(12).
8. **Decomposition cost value calculation:** For each child node, calculate the convex hull for the two polygons obtained due to

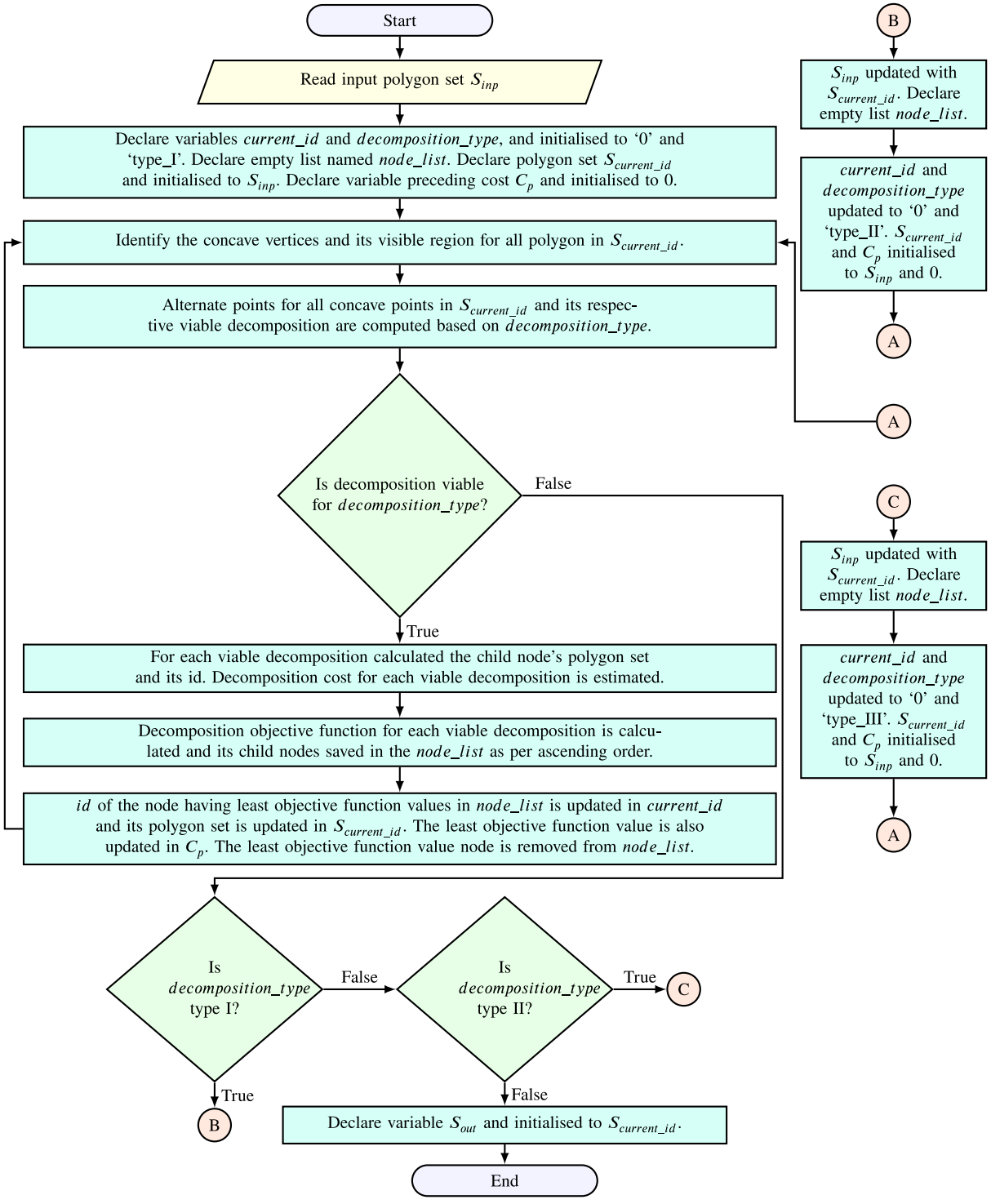


Fig. 15. Process flow chart of Dijkstra based recursive polygon decomposition.

decomposition in the child node's polygon set, as explained in Section 2.1.2. Then for these two polygons, convex hull widths are computed as explained in Section 2.1.1. Similarly, the convex hull width of the parent polygon is also found. The decomposition cost value for each child node is calculated using hulls width as per Eq. (14).

9. **Decomposition objective function calculation:** The decomposition objective function value for the child nodes from the *current_id* act as a parent are computed as per Eq. (15). The child nodes and their objective function values are stored alongside already available details in the *node_list*. Based on the decomposition objective function value, the *node_list* is sorted in ascending order. The node in the *node_list* having minimum objective function value is taken, and the respective *id* is updated in the *current_id*. The minimum objective function value is updated in the preceding cost C_p. The polygon set for *current_id* is also updated accordingly in $S_{current\_id}$. Then, the child node having minimum objective function value is removed from the *node_list*.

10. **Recursive calculation:** Then steps 2 to 9 are recursively repeated till there is no possibility of the type I decomposition or all the decomposed polygons in the S_{curent_id} are convex. Then, the polygon set S_{curent_id} is considered as an input for the next tree built using type II decomposition. So, S_{inp} is updated with S_{curent_id}.
11. **Decomposition type updation for type II:** The *current_id* and the *decomposition_type* is updated as 0 and 'type_II'. Again, the empty list named *node_list* is defined. Declare the variable preceding cost C_p and initialised to zero. Then, steps 2 to 9 are recursively repeated till there is no viability for type II decomposition. Then, the polygon set S_{curent_id} is considered as an input for the next tree built using type III decomposition. So, S_{inp} is updated with S_{curent_id}.
12. **Decomposition type updation for type III:** The *current_id* and the *decomposition_type* is updated as '0' and 'type_III'. The empty list named *node_list* is defined. Declare the variable preceding cost C_p and initialised to zero. Then, steps 2 to 9 are recursively repeated till there is no viability for type III decomposition. Then defined the output polygon set S_{out} and is updated with the polygon set at S_{curent_id}.

The recursive decomposition operation is programmed in Python 3.8 (Python Software Foundation, 2021) and applied to the concave polygon shown in Fig. 10 The result of this simulation is shown in Fig. 16.

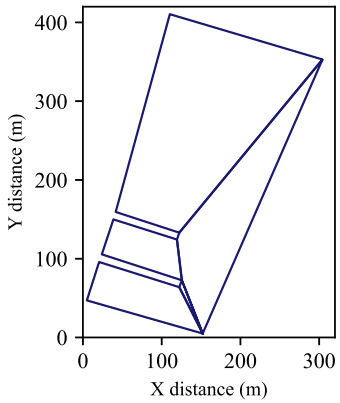


Fig. 16. Graph showing the output of the recursive decomposition operation on the concave polygon.

3.2. Dijkstra based polygon unification

The final output polygon set obtained from DA based recursive polygon decomposition contains the convex polygons. It is viable to implement BF based CPP in every convex polygon, but more polygons are available; this makes the autonomous vehicle transfer from one to another polygon many times during operation. As a result, the major path line orientation must be updated in the autonomous vehicle mission planner often and increases the transition length between the polygons, affecting overall trajectory length. Also, the particular polygon may be small in area to do mission separately. It is essential to reduce the number of polygons, and the polygons should be reasonably large. So, in this unification step, the convex polygons in the polygon set obtained in the decomposition step are merged such that BF based CPP implementation is feasible. It is discussed in Section 2.1.2 that the BF pattern is viable in some concave polygons. During this unification operation, the resultant polygon's convex hull width direction may not be feasible for BF based CPP implementation. But the BF based CPP can be implemented in polygon's other V–E span directions, especially one whose length is least among the feasible spans leads to minimum turn as per Eq. (1). There has to be a tradeoff maintained between the summation of each polygon's least length V–E span, which signifies the number of turns and the minimum number of polygons. In this Section, the polygons count is reduced meanwhile sustain a better number of turns summation when BF based CPP is implemented in each polygon. The formulation and procedure to attain this objective is given in Sections 3.2.1 and 3.2.2.

3.2.1. Polygon unification formulation

Generally, one polygon is considered a neighbour to the other polygon only if they have common edges. These common edges are classified as continuous and discontinuous common edge relations and are shown in Figs. 17(a–b).

In Fig. 17(a), the polygons P_1 and P_2 have a continuous common edge formed by vertices v_3 and v_4. The common edges can be more than one also, but these common edges are sequentially closer to one another when both polygon edges are represented in the anticlockwise direction. In Fig. 17(b), the polygons P_1 and P_2 have two common edges formed by vertices $v_3 - v_4$ and $v_6 - v_7$, which forms the discontinuous common edge relation. Also, it can be inferred that these two edges are not sequentially closer to one another when both polygon edges are represented in the anticlockwise direction. The discontinuous common edge relation happens only when some other polygon is inscribed in between two polygons. Fig. 17(b) shows that the polygon P_3 inscribed between the polygon P_1 and P_2 creates a discontinuous common edge relation.

The discontinuous edge relation is not possible in the final polygon set obtained from recursive decomposition, and it can be inferred by understanding the nature of the polygon edge. All polygon edges in the final polygon set from decomposition operation can be classified into the physical and virtual edge and are shown in Fig. 18.

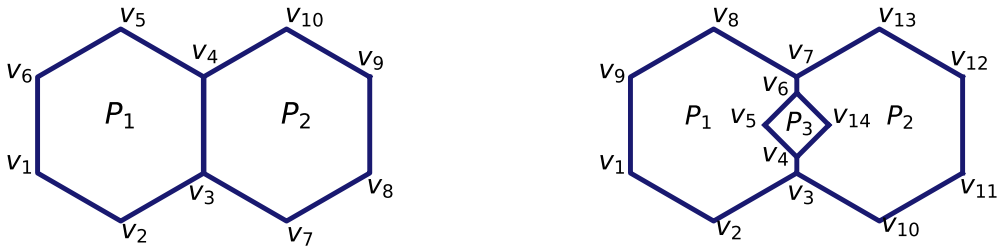
The physical edges are the edges of the initial input polygon because they have physical constraints such that the vehicle should not cross it, as shown in Fig. 18. The virtual edges are obtained from edges drawn for the decomposition operation because the vehicle can cross these edges, and there is no physical constraint with it. In Fig. 18, the virtual edge is generated due to decomposition edge $v_1 - v_5$. In recursive decomposition, whenever resultant decomposed polygons are estimated, they have at least one physical edge because the concave vertex always lies in the physical edge. As far as discontinuous common edge relation considered, all the edges of the inscribed polygon must be a virtual edge that contradicts that the decomposed polygon has at least one physical edge. So, it can be concluded that each polygon in the final polygon set obtained from the recursive decomposition either have common edge relation with other polygons in the same polygon set or none.

The polygons are unified by removing common edge relation, and the polygon P_1 and P_2 shown in Fig. 17(a) has common edge v_3 and v_4. Removing this common edge generates the unified polygon.

Similar to the decomposition operation, when a single unification is planned on the final polygon set obtained from recursive decomposition operation, the numerous ways it can be unified and the unification operation are done so that the BF based CPP implementation is possible in the unified polygon. Recursive implementation of unification operation also generates a tree similar to the decomposition tree, as shown in Fig. 19 for an arbitrary polygon set.

Tree definition and nomenclature

The general tree shows the recursive application of unification operation leads various polygon sets is shown in Fig. 20. The unification operation which generates the unified polygon on which BF based CPP implementation is viable is only considered for the tree generation. The polygon set, polygon vertices, node identification variable *id* definitions considered for the decomposition tree is also adopted for the general unification tree.



(a) (b)

Fig. 17. Schematic diagram showing the (a) continuous and (b) discontinuous common edge relation between polygons.

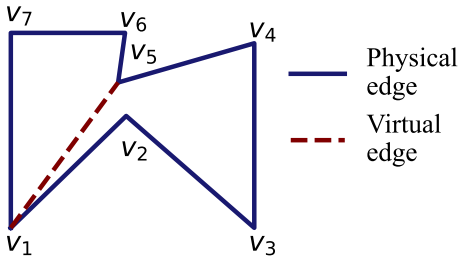


Fig. 18. Schematic diagram showing the physical and virtual edge in the polygon.

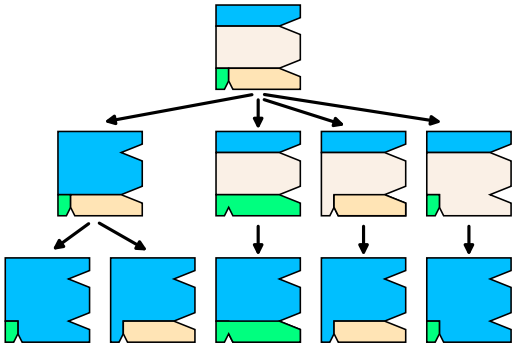


Fig. 19. Schematic diagram showing the recursive application of unification operation on an arbitrary polygon set.

A node whose identification variable *id* is 0 and has a polygon set contains decomposed convex polygons obtained from the recursive decomposition operation present at zero-level state. Computed the various possible common edge relations essential for unification operation for the polygons in the polygon set at node 0. Each of these relations unifies two polygons in the node 0 and generates one unified polygon. Only some operations are considered viable out of all unification operations derived from the calculated common edge relation. These operations are evaluated for feasibility based on three conditions and are listed below.

1. The existence of a common visible region for the unification operation's resultant unified polygon are calculated as explained in Section 2.1.2. If a common visible region exists, then the respective unification operation clears the first condition.
2. The V–E spans for the unified polygon's convex hull are calculated and checked for its normal direction presence in the common visible region as briefed in Section 2.1.2. If at least one V–E span lies in a common visible region, then the respective unification operation satisfies the second condition. V–E span is considered for viability checking because width lies in the V–E span.
3. Defined a variable λ which is the difference between the least span length (span clears first two viability condition) summation of two polygons undergoing unification operation and the least span length of a single polygon evolved from the unification operation. If the variable λ is less than zero, then the particular operation reduces the net span length, which is good. When BF based CPP is implemented, the number of turns is reduced in this unified polygon compared to its previous polygons. But it is also possible λ can be greater than zero, which increases the least span length in the unified polygon. So, there has to tradeoff maintained between the polygon count and least span length, which affects the number of turns. So, defined a span length limiting condition σ and assigned the specific value to it. If $\lambda < \sigma$, then the respective unification operation satisfies the third condition.

Out of all unification operations calculated at node 0, the operations that satisfied the three viability conditions are considered a viable unification operation. These viable unification operations are only considered for generating the tree. The node on which unification operation is done is the parent, and the node attained due to respective unification is the child. The viable unification operation for the polygon set at node 0 is calculated and is shown in Fig. 20 as a branch. Each branch leads to nodes in a first-level state. The node in a first-level state contains a polygon set that consists of a unified polygon from the respective operation and the polygon in the parent (node '0') that are not unified. Each child is considered a new parent. The tree is recursively expanded until it reaches a node where the further application of viable unification operation becomes void, as shown in Fig. 20.

The unification operation is mathematically expressed in Eqs. (16)–(17).

$$\mathbf{V}(P_{cid}^{ii}) = \mathbf{U}_M \begin{bmatrix} \mathbf{V}(P_{pid}^{ii}) \\ \mathbf{V}(P_{pid}^{jj}) \end{bmatrix} \tag{16}$$

see Eq. (17) given in Box I.

Where $\mathbf{V}(P_{pid}^{ii}) \in \mathbb{R}^{L3\times 2}$ and $\mathbf{V}(P_{pid}^{jj}) \in \mathbb{R}^{L6\times 2}$ are the vertices of the polygons P_{pid}^{ii} and P_{pid}^{jj} in the parent node whose node identification variable id is denoted as pid, and the respective local polygon identification number is indicated by ii and jj such that $ii < jj$. $L3$ and $L6$ are the numbers of vertices in the polygons P_{pid}^{ii} and P_{pid}^{jj}. The common edges and their respective vertices are found in the polygons P_{pid}^{ii} and P_{pid}^{jj}. The row numbers of the vertices contributing common edges in P_{pid}^{ii} are found, and the minimum and the maximum value row number is considered as $L1$ and $L2$. The row numbers of the vertices contributing common edges in P_{pid}^{jj} are found, and the minimum and the maximum value row number is considered as $L4$ and $L5$. The vertex available at row number $L1$ in P_{pid}^{ii} and $L5$ in P_{pid}^{jj} should be the same, and if not same, then the polygon P_{pid}^{jj} vertices should be represented in some other form such that considering

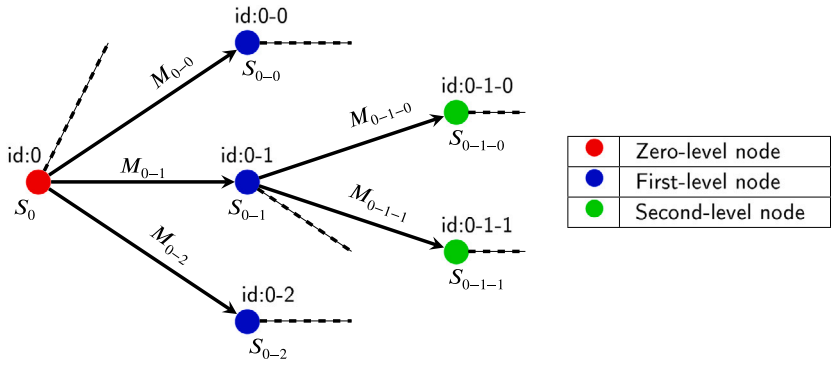


Fig. 20. Schematic diagram showing the sample tree represents the numerous polygon set obtained at various nodes due to the recursive polygon unification operation.

$$\mathbf{U}_M = \begin{bmatrix} \mathbf{I}_{(L1-1)\times(L1-1)} & \mathbf{0}_{(L1-1)\times(L2-L1+1)} & \mathbf{0}_{(L1-1)\times(L3-L2)} & \mathbf{0}_{(L1-1)\times L4} & \mathbf{0}_{(L1-1)\times(L5-L4-1)} & \mathbf{0}_{(L1-1)\times(L6-L5+1)} \\ \mathbf{0}_{(L6-L5+1)\times(L1-1)} & \mathbf{0}_{(L6-L5+1)\times(L2-L1+1)} & \mathbf{0}_{(L6-L5+1)\times(L3-L2)} & \mathbf{0}_{(L6-L5+1)\times L4} & \mathbf{0}_{(L6-L5+1)\times(L5-L4-1)} & \mathbf{I}_{(L6-L5+1)\times(L6-L5+1)} \\ \mathbf{0}_{L4\times(L1-1)} & \mathbf{0}_{L4\times(L2-L1+1)} & \mathbf{0}_{L4\times(L3-L2)} & \mathbf{I}_{L4\times L4} & \mathbf{0}_{L4\times(L5-L4-1)} & \mathbf{0}_{L4\times(L6-L5+1)} \\ \mathbf{0}_{(L3-L2)\times(L1-1)} & \mathbf{0}_{(L3-L2)\times(L2-L1+1)} & \mathbf{I}_{(L3-L2)\times(L3-L2)} & \mathbf{0}_{(L3-L2)\times L4} & \mathbf{0}_{(L3-L2)\times(L5-L4-1)} & \mathbf{0}_{(L3-L2)\times(L6-L5+1)} \end{bmatrix} \tag{17}$$

Box I.

some other vertex in the first row followed by subsequent vertices in an anticlockwise direction until mentioned condition for $L1$ and $L5$ satisfies. $\mathbf{U_M} \in \mathbb{R}^{(L6+L3+L1-L2+L4-L5)\times(L3+L6)}$ is the unification operation matrix. $V(P_{cid}^{ii}) \in \mathbb{R}^{(L6+L3+L1-L2+L4-L5)\times 2}$ is the vertex of the polygon P_{cid}^{ii} obtained due to unification operation.

The polygon set for the child node contains the polygon obtained due to the unification P_{cid}^{ii} and the other polygons from the parent node, which are not responsible for unification operation. The polygon set for the child node obtained in the unification operation is expressed as Eqs. (18)–(23).

$$S_{cid} = S_{uu} \cup \{ P_{cid}^{ii} \} \tag{18}$$

$$S_{uu}^{1} = \{ P_{cid}^{k} \mid k \in \mathbb{W},\ k < jj,\ k \neq ii \} \tag{19}$$

$$P_{cid}^{k} = P_{pid}^{k} \quad \text{such that} \quad k \in \mathbb{W},\ k < jj,\ k \neq ii \tag{20}$$

$$S_{uu}^{2} = \{ P_{cid}^{k} \mid k \in \mathbb{W},\ k \geq jj,\ k < N_p^{pid} - 1 \} \tag{21}$$

$$P_{cid}^{k-1} = P_{pid}^{k} \quad \text{such that} \quad k \in \mathbb{W},\ k > jj,\ k < N_p^{pid} \tag{22}$$

$$S_{uu} = S_{uu}^{1} \cup S_{uu}^{2} \tag{23}$$

S_{uu} is the polygon set that contains the polygons in the parent node that are not used for unification operation and N_p^{pid} is the number of polygons in the parent polygon set.

Unification cost and objective function definition

The unification step aims to reduce the polygon count and maintain better polygon widths summation for resultant polygons obtained in this step. This objective is taken by defining the unification cost function as per Eq. (24) for every unification operation.

$$M_{cid} = D_{least}(P_{cid}^{ii}) - D_{least}(P_{pid}^{jj}) - D_{least}(P_{pid}^{kk}) \tag{24}$$

Where M_{cid} is the unification cost function for the operation implemented to attain node whose identification is cid from their parents and $D_{least}(P_{cid}^{ii})$ is the minimum length feasible V–E style span present at polygon P_{cid}^{ii} whose local polygon identification number is ii. $D_{least}(P_{pid}^{jj})$ and $D_{least}(P_{pid}^{kk})$ are the least length feasible V–E style span present at parent polygons of the unification operation whose node identification variables are denoted as pid, and the local polygon identification numbers are denoted as jj and kk. Suppose for a single node, multiple feasible unification operations are possible, but the operation has the least M_{cid} considered to be better than other unification operations because minimum span signifies the minimum number of turns if BF based CPP is implemented compared to other unification operations. This unification cost function is defined on the tree shown in Fig. 20. Then, the unification's objective function is defined based on the unification cost function as per Eq. (25). Minimising the unification's objective function leads to a node in the tree that has reduced polygon count with a reasonably better number of turns when BF based CPP is implemented.

$$H_{id} = M_p + M_{id} \tag{25}$$

Where H_{id} is the objective function value is to attain a node whose identification is id from node 0. M_p is the preceding unification cost, which is the summation of the unification cost for the sequence of unification operations to reach the current node's parent from the node 0 at a zero-level state. Based on this objective function, the tree is solved for minimisation by the DA.

3.2.2. Solving the formulation

As per the formulation defined in Section 3.2.1, the steps to reduce the polygons count and attain resultant polygons whose least V–E span length summation better are explained below and is shown in Fig. 21.

1. **Initialisation :** A polygon set $S_{inp} = \{ P^k \mid k \in \mathbb{W} \ \ and \ \ k < N_p^{inp} \}$ that holds the decomposed polygon from the recursive decomposition step is provided as an input. The N_p^{inp} is the number of polygons present in the input polygon set. Then defined the variable $current\_id$. The $current\_id$ is initialised to the id of the node present at zero-level, which is 0. The polygon set for the $current\_id$, S_{curent_id} is defined and initialised to the input polygon set S_{inp} because $current\_id$ is 0. Define the variable unification preceding cost M_p and initialised to zero. The span limiting condition σ is defined and initialised to any value as discussed in Section 3.2.1 as per the need. Define the empty list named $node\_list$.
2. **Feasible unification operation identification:** The common edge relation for the polygon set at the current node. For each

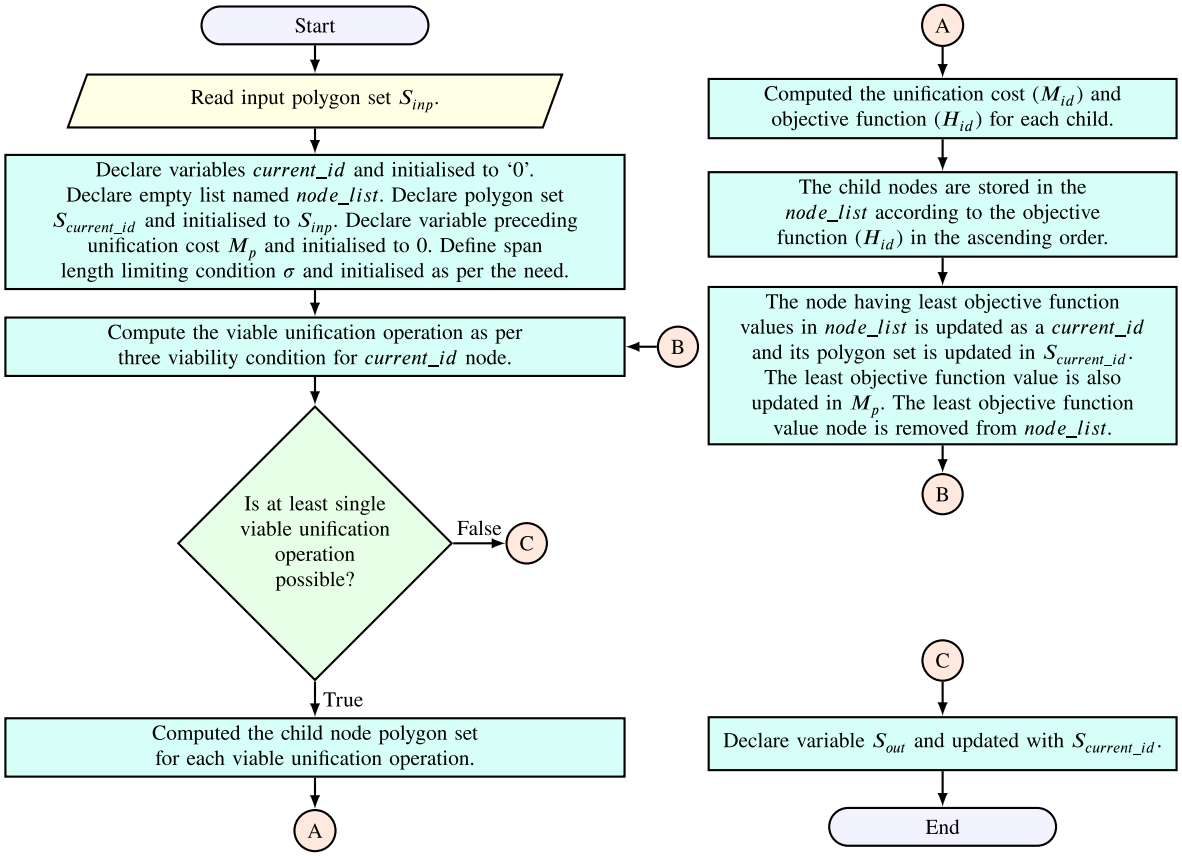


Fig. 21. Process flow chart of Dijkstra based recursive polygon unification.

common edge relation, three viability conditions as discussed in Section 3.2.1 are tested, and the viable unification operations are computed.

3. **Computing viable polygon set:** If at least one viable unification operation exists, then the respective polygon sets for the next nodes (child nodes) from *current_id* (parent node) are calculated as per Eqs. (16)–(23). If none of the viable unification operations exists for further tree expansion, then go to step 7.
4. **Merger cost value calculation:** The merger cost (M_{id}) for the child nodes from *current_id* are calculated as per Eq. (24) and saved.
5. **Objective function calculation:** The objective function (H_{id}) for the child nodes from the *current_id* is evaluated. These child nodes with their objective values are stored in the search list alongside already available data. The node having the least objective function value is found, and their respective *id* is updated on the *current_id*. The respective least objective function value is updated in the preceding cost M_p. Then, the node's details having the least objective function are removed from the *node_list*.
6. **Recursive unification:** Steps 2 to 5 are recursively repeated till there is no possibility for viable unification operation.
7. **Optimised polygon set:** The polygon set S_{out} is defined and updated with the polygon set $S_{current\_id}$ of the *current_id*. The polygon set S_{out} is considered to be the final optimised polygon set.

The recursive decomposition operation is programmed in Python 3.8 and applied to the decomposed polygons shown in Fig. 17, and the result of this simulation is shown in Figs. 22(a–b) for different span length limiting condition σ equal to 0 and ∞ m. It is also inferred from Fig. 22(a), no unification operation is viable for $\sigma = 0$ m because input decomposed polygon and output of the recursive unification are the same.

3.3. Feasible grid points generation

The BF based CPP is applied on polygons in the polygon set obtained after the recursive decomposition and unification operation needs the grid point or waypoint. The viable V–E style span and its orientation for each polygon in the polygon list are found as per Section 2.1. Each polygon has two kinds of edges, physical edge and virtual edge, as discussed in Section 3.2.1. The CPP generate sets of waypoints in the polygons for guiding the vehicle. As far as certain autonomous vehicle applications such as AUV and AGV been concerned, there has to be a certain perpendicular distance to maintain from the physical edge to avoid the collision. The distance to be maintained is the grid distance which is considered for waypoint generation. So, another polygon called filter polygon is generated inside each polygon, similar to the concentric polygon. This filter polygon maintains half of the grid distance from the virtual edge and full grid distance from the physical edge, and it ensures that the two filter polygon on two sides of the virtual edge is at full grid distance. The filter polygon for the polygons in the polygon set shown in Fig. 22(b) is shown in Fig. 23.

The filter polygon is required only for some autonomous vehicles, and for other vehicles such as UAV can cross even the physical boundary. In this case, the filter polygon is not needed, and the original polygon itself acts as a filter polygon. The sweep line direction and respective normal major pathline direction are calculated for each polygon such that the number of turns is lesser in each polygon depends on the V–E style span, and then the grid points are generated along this direction as per the grid size specified. Each grid point or waypoint has four neighbour points, two along the sweep line and two along the

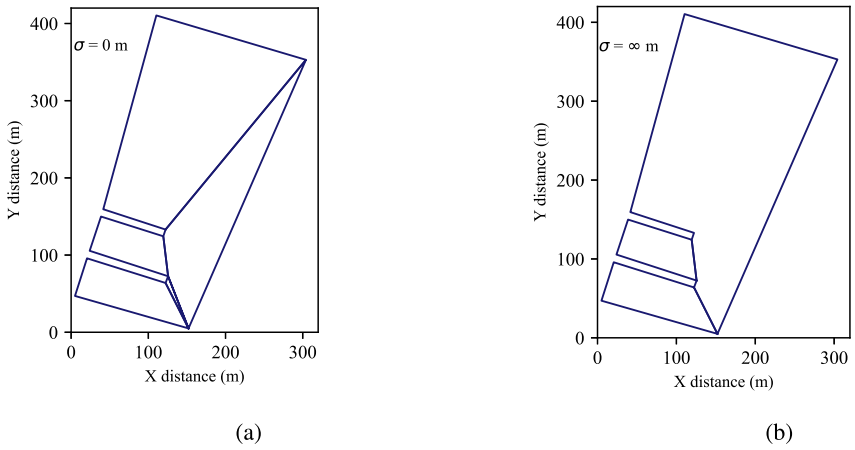


Fig. 22. Graph showing the output of the recursive unification operation on the decomposed polygons for (a) σ=0 and (b) $\sigma = \infty$ m.

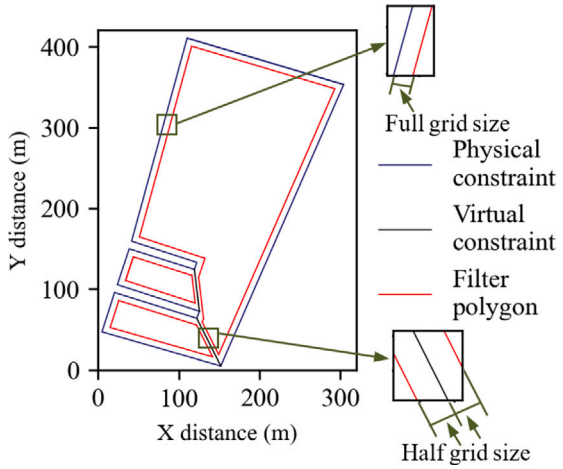


Fig. 23. Graph showing the filter polygon for each polygon in the polygon set.

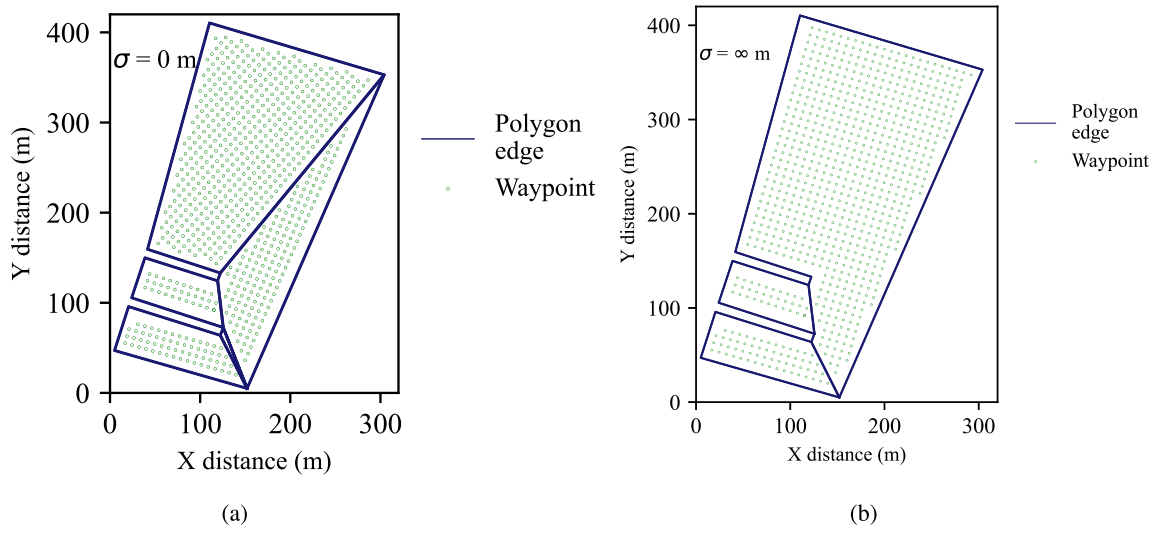


Fig. 24. Graph showing the waypoints generated in each polygon belong to polygon set obtained after recursive decomposition and unification operation for span limiting conditions (a) 0 and (b) ∞ m considering the grid size of 8 m.

major path line direction. In this grid points generation, the distance between a certain waypoint and its neighbour is at grid size either in a sweep or major path line direction. Then the grid points that lie inside the filter polygon are selected. In this work, python's matplotlib library (Matplotlib, 2007) has a special feature 'contains points' utilised for this operation. The gridpoint or waypoint for the polygon set in Figs. 22(a–b) are generated and is shown in Figs. 24(a–b).

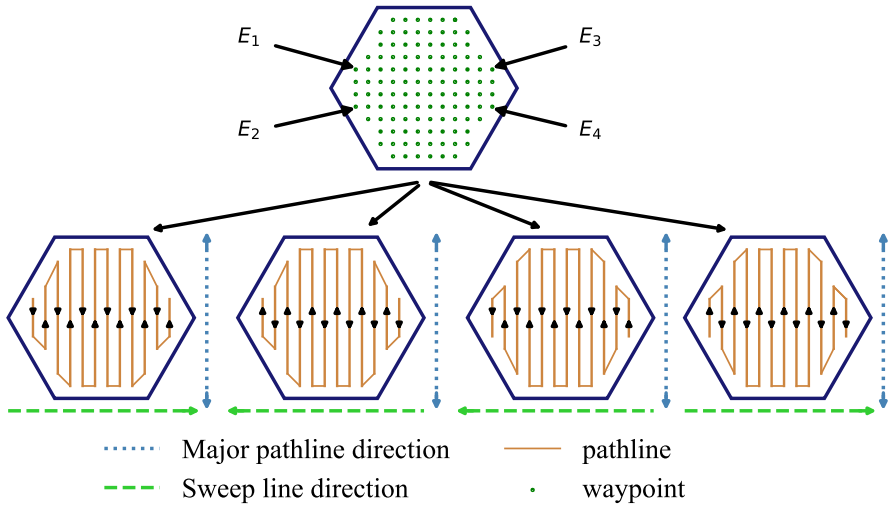


Fig. 25. Schematic representation showing four viable BF based CPP for an arbitrary polygon.

3.4. Ant colony optimisation of overall trajectory length

The procedure for the BF pattern implementation on the grid points or waypoint generated as per Section 3.3 is discussed in this Section. The manner in which the BF pattern is implemented and the order in which polygons are covered during an inspection by the vehicle affects the overall operation trajectory length. In this work, finding the optimised overall trajectory is mathematically formulated and is solved using the ACO.

3.4.1. Formulation

The BF pattern can be implemented on the polygon in four ways depending on the extreme grid points. Fig. 25 shows the arbitrary polygon and its inscribed waypoints. The extreme points for the grid points are E1, E2, E3 and E4 are shown in Fig. 25. Generally, any waypoint has four neighbours, two along the sweep line direction and two along the major pathline. But the extreme points are the one which has only two or one neighbours along sweep line or major pathline direction and similar extreme points are shown in Fig. 25.

The BF pattern is implemented in four ways considering each extreme point as the starting point and the respective major path line direction. So the viable BF based CPP implementations are E1 to E4, E4 to E1, E2 to E3 and E3 to E2. In Fig. 25, the number of major path lines is twelve; it is also observed that starting and ending points are on the same sides, such as E1 and E3, for an even number of major path lines. If the number of major path lines is odd, then the starting and ending points are not on the same side.

Suppose the polygon set obtained after recursive decomposition and unification operation has N number of polygons. Then, the number BF based CPP configuration possible for the polygon set is 4N. For inspection operation, the vehicle has to cover all the polygon at once. So, it is necessary to choose N BF configuration out of 4N BF configuration such that BF configuration is chosen from each polygon, not more than one and N BF configuration covers all polygons in the polygon set obtained after recursive decomposition and unification.

Once the autonomous vehicle covers one polygon and it shifts to another polygon for inspection. This transition happens between the last waypoint of the covered polygon's CPP to the first waypoint of another polygon's CPP. The length of travel distance between these waypoints is called transition distance.

The overall trajectory length for the polygon set is the summation of CPP length for each polygon in the polygon set and each transition distance between the polygons while shifting from one polygon to another polygon for operation. The overall trajectory length has to be optimised to reduce the system's energy and the operation's time. Optimising the transition length component in overall trajectory length decreases the overall trajectory length. This transition length depends on the order in which the polygons in the polygon set are inspected. The transition length is the distance travelled by the vehicle from the covered polygon CPP's last waypoint to the next polygon CPP's first waypoint.

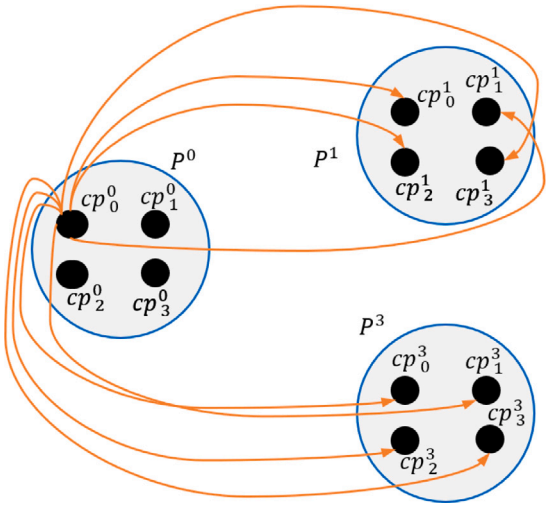


Fig. 26. Schematic diagram showing the possible transition path for single BF configuration in a specific polygon to other polygons.

For three polygons, each one with four BF based CPP configurations has multiple transition possibilities, shown in Fig. 26. In Fig. 26, polygons are defined as per Eq. (2). BF based CPP configuration for each polygon is defined as cp^{pn}_{bf}. The bf denotes the identification number refers to four BF based CPP configurations in a single polygon similar to the configuration shown in Fig. 26. bf can be 0,1,2 or 3. pn is the local polygon identification number for the polygons in polygon set obtained after recursive decomposition and unification operation.

Fig. 26 shows a viable transition distance for a single BF based CPP configuration in a single polygon because it is difficult to express all the viable transition distances. So, the all the viable transition for the case in Fig. 26 is shown in Fig. 27.

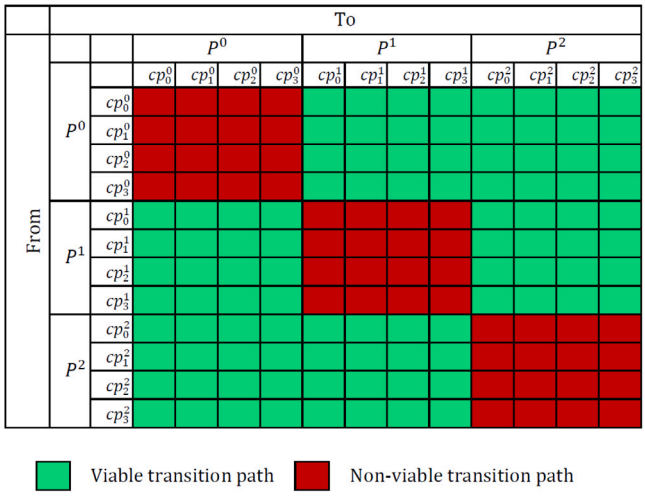


Fig. 27. Schematic diagram showing all possible transition paths for three polygons.

In Fig. 27, the transition path between BF based CPP configurations in the same polygon is not possible, so it is considered a non-viable transition path. The objective is to find an order of polygons that optimises overall trajectory length such that the vehicle visits each polygon once. This objective is similar to well known Travelling Salesman Problem (TSP). In TSP, a salesman has to visit all the cities under consideration once, and the total travel distance is expected to be less. TSP problem's city is equivalent to polygon and its BF based CPP configuration. TSP's paths between cities are similar to transition paths between various BF based CPP configurations, as shown in Fig. 27. But the problem discussed in this Section differs from TSP in certain aspects listed below, which poses the challenge while solving the problem.

1. In the TSP problem, the distance between any cities are the same in both directions, but transition distance is not always the same in both direction for a viable path from one BF configuration to another BF configuration because the last waypoint of one polygon and the first waypoint of another polygon which varies when considered both directions of movement.
2. In TSP, a city visited means a single node reached, but for the polygon case, the polygon is considered visited if one of the BF configurations out of four configurations is visited.

The number of viable transition N^{trans} paths for N polygons is given by Eq. (26).

$$N^{trans} = 4N \times 4(N-1) \tag{26}$$

Out of N^{trans} transition path, only $N-1$ transition paths that cover all polygons need to be calculated such that summation of those transition paths distance is least, which is the optimal solution. For three polygons, the N^{trans} is 96; out of 96 paths, only two paths whose transition distance summation least needs to be found. Even for simple three polygons, a huge number of 96 viable transition paths needs to be considered to find the group of transition paths whose distance summation is the least. This problem is a kind of combinatorial optimisation problem because, in this problem, an optimal solution is found from the finite set of feasible solutions. Finding the solution for this problem using brute force calculation requires tremendous computational power as the number of polygons increases. Also, it inferred from the literature (Li et al., 2008; Ariyasingha and Fernando, 2015), similar TSP is solved efficiently and rapidly by ACO due to its metaheuristic nature. So, the ACO (Li et al., 2008) algorithm is selected for solving the problem in this work.

The transition distance between polygon has to be expressed mathematically for solving the problem, but it is not easy because the vehicle may not follow the straight path for transition. So, the Euclidean distance between the last waypoint in the covered polygon and the first waypoint in the next polygon to be shifted can be considered a transition distance for solving. This assumption is also considered as a heuristic for the problem.

The ACO depends on the behaviour followed by the ant while it is searching for the food. The ant always deposits pheromone (a chemical component) while travelling, and this deposition evaporates continuously. While searching for food, a group of ants comes out of the colony. These ants go in a different direction by depositing pheromone on the way, and once any of the ants found the food and returns back to the nest by sniffing the already deposited pheromone. The path travelled by the ant that found the food has heavy pheromone deposition because pheromone is deposited again in the same path while returning to the nest. So, when the next group of ants starts searching for food, they mostly choose the path has heavy pheromone deposition, and some ants may go in other directions depending on the pheromone level in the path. These pheromones also evaporate continuously. After some time, the path where most ants travelled has heavy pheromone deposition, so all the ants follow this path mostly. Also, for the same food location, the ant can also reach in many ways. Since the pheromone evaporates continuously, leading to the path whose length is shorter has heavy deposition than the longer path because, in the longer path, two consecutive pheromone deposition takes time, and most of them evaporate. This behaviour optimises the distance between food and nest location. This ant's behaviour can be computationally modelled to solve the optimisation problem. The ACO algorithm for solving the polygon order, which reduces the polygon transition length, is explained in Section 3.4.2. The ACO methodology for this paper is adapted from Li et al. (2008) and are modified according to the problem discussed in this paper.

3.4.2. Solving the formulation

ACO procedure for solving the formulation explained in Section 3.4.1 are given below and shown in Fig. 28.

1. **Initialisation:** Read the input polygon set from recursive decomposition and unification and its four BF based CPP configuration. Defined and initialised the number of ants N_{ants} and number of polygons N. Define the initial pheromone constant $\gamma > 0$ and initialised as per the need. The pheromone matrix $\tau \in \mathbb{R}^{4N\times4N}$ is defined, and their rows and columns represent the polygon with their BF based CPP configuration cp_{bf}^{pn} which is similar to row and column-wise BF based CPP configuration indicated in Fig. 27.

   $$\tau = \gamma \times U_{4N\times4N} \tag{27}$$

   Where U is the matrix whose elements are one and its size is $4N \times 4N$, then elements correspond to the same polygon in row and column irrespective of BF configuration for pheromone matrix are initialised to zero. Pheromone $\alpha(>0)$ and transition distance $\beta(>0)$ weightage parameters are defined, which are used in step 2 for transition path selection. The pheromone evaporation and deposition rate ρ and ζ are defined and initialised. Out of N_{ants}, the first ant is chosen as a current ant. The number of iterations $N_{iteration}$ for the ACO is defined and initialised. Define the variable *current_ant* and *ite* and initialised to 1 and 1. *current_ant* and *ite* are the identification variable for ant and iteration while running the loop. Define an empty list *best_tour* for saving the best tour made by ants in subsequent steps.
2. **Next transition path selection:** The current ant corresponds to the *current_ant* variable is randomly placed on any polygon and respective polygon's any BF configuration, which is also considered a location for current ant in path diagram drawn

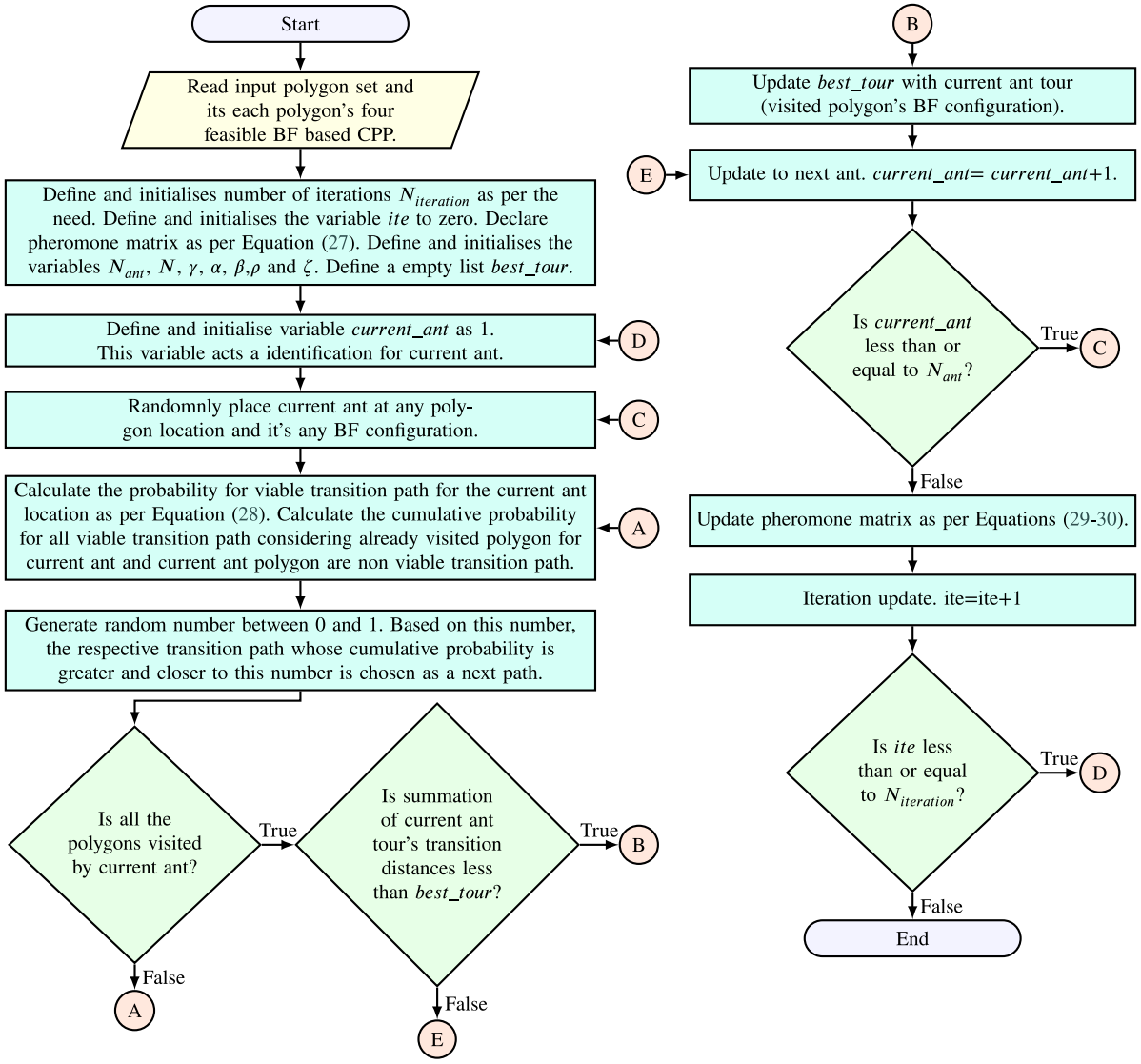


Fig. 28. Process flow chart of ACO algorithm for polygons transition distance optimisation.

similar to the diagram shown in Fig. 26. From the current ant location calculated the Euclidean transition distance for all viable transition paths as discussed in Section 3.4.1. The probability for moving along any viable transition path is calculated as per Eq. (28).

$$ {}^{pn1}_{bf1}Pr^{pn2}_{bf2} = \frac{\left({}^{pn1}_{bf1}Ph^{pn2}_{bf2}\right)^{\alpha}\left(\frac{1}{{}^{pn1}_{bf1}td^{pn2}_{bf2}}\right)^{\beta}}{\sum_{viable}\left({}^{pn1}_{bf1}Ph^{pn2}_{bf2}\right)^{\alpha}\left(\frac{1}{{}^{pn1}_{bf1}td^{pn2}_{bf2}}\right)^{\beta}} \tag{28} $$

Where $pn1$ is the local polygon identification number corresponds to the current polygon where ant lie and whose BF configuration is identified as $bf1$. The notation $pn2$ is the local polygon identification number corresponding to the next viable polygon for ant shifting and whose BF configuration is identified as $bf2$. Pr is the probability of the transition path from the polygon $pn1$ and BF configuration $bf1$ to the polygon $pn2$ and BF configuration $bf2$. Ph is the pheromone value taken from the pheromone matrix for moving from polygon $pn1$ and $bf1$ to $pn2$ and $bf2$. The td is the Euclidean transition distance. Using Eq. (28), the probability is found for all viable transition paths from the current ant location.

The cumulative probability for these viable transition paths is found. Randomly a number is generated between 0 and 1. The path whose cumulative probability is greater and closer to this number is found based on this number. This path is chosen as the next path for the current ant. This path leads to a new polygon and its BF configuration similar to the example as shown in Fig. 26 and is chosen as the current location of the ant. Once an ant visited a polygon regardless of BF configuration, all BF configuration in that polygon is considered non-viable from other polygons after that. In this step 2, the constants α and β decides the weightage given to pheromone and distance when the probability of choosing a viable path from the current ant location.

3. **Recursive calculation:** Step 2 is repeated for the current ant until the current ant visits all the polygons once. If defined *best_tour* is empty, then the current ant tour is updated in the list *best_tour*; else, the total transition distance summation for the current ant tour is compared with the best tour. If the current ant transition length summation is lesser than the best tour, then the current ant tour is updated as the best tour.

4. **Next ant:** Then, the current ant is updated with another ant out of N_{ants}, and the variable *current_ant* is incremented by unit. Then, steps 2 and 3 are repeated for the current ant. Step 4 is repeated until all ants completed the tour.
5. **Pheromone matrix update:** The pheromone matrix must be updated after all ants visit all the polygons at once. The pheromone matrix is updated as per Eqs. (29)–(30).

$$\tau_{IJ} = (1-\rho)\,\tau_{IJ} + \sum_{k=1}^{N_{ants}} \frac{\zeta}{L_k} v_k^{IJ} \tag{29}$$

$$v_k^{IJ} = \begin{cases} 1 & if\ k^{th}\ ant\ visited\ path\ from\ I\ to\ J \\ 0 & else \end{cases} \tag{30}$$

Where τ_{IJ} is the pheromone matrix's respective pheromone element represents the pheromone level in the path from one polygon's BF configuration in a row denoted as I to the other polygon's BF configuration in a column denoted as J. The row and column identifier for the pheromone matrix I and J are used to represent various BF configurations in a polygon previously referred as cp_{bf}^{pn} in the general form, which is also similar to the one shown in Fig. 27. L_k is the length of the tour made by the kth ant. The variable v_k^{IJ} is one if the kth ant visited the path else zero, and this is to ensure that only a visited ant deposit pheromone. It is also evident from Eqs. (29)–(30), the shorter path travelled and deposited more pheromone.

6. **Next iteration:** The variable *ite* is incremented by unit, and the *current_ant* variable is updated as one. Steps 2 to 6 are repeated until *ite* is greater than $N_{iteration}$ or until $N_{iteration}$ iterations are done. After all, iterations are done, the group of transition paths stored in *best_tour* is the optimal solution.

The ACO based polygon order optimisation leads to better overall trajectory length by reducing its polygon transition length components which are applied to the polygon set shown in Figs. 22(a–b). The ACO based optimisation procedure in this paper is programmed in Python 3.8, and the resultant optimised BF based CPP are shown in Figs. 29(a–b). In Fig. 29(a), the BF based CPP for the polygon set whose span limiting condition zero is shown, and also the output of ACO instructs the do operation in the following order: sector 1, 2, 3 and 4. In Fig. 29(b), the BF based CPP for the polygon set whose span limiting condition infinity is shown, and also the output of ACO instructs the do operation in the following order: sector 1, 2 and 3. These recommended order by ACO reduces the unnecessary transition length. The numbers of turns for span limiting condition zero and infinity in Figs. 29(a–b) are 63 and 95. The total Euclidean polygon transition distance for the span limiting condition zero and infinity cases is shown in Figs. 29(a–b) are 79.14 and 56.92 m. The CPP trajectory length summation of each polygon for the span limiting condition zero and infinity cases in Figs. 29(a–b) are 6130.96 and 6561.49 m. The polygon transition distance for span limiting condition zero is less than infinity in this case, but it is not true always. The ACO steps in the proposed algorithm try to reduce total polygon transition distance regardless of span limiting condition. The ACO parameters α, β, γ, ρ, ζ and iteration for outputs are shown in Figs. 29(a–b) are 15, 20, 0.1, 0.2, 1 and 10.

4. Performance and parameter analysis of proposed coverage path planner

In this Section, the proposed CPP is implemented on Chennai's Kasimedu Fishing Harbour case study, and then the performance parameters of the proposed algorithm are studied through the case study. Later the proposed CPP is compared with the published result for performance.

4.1. Implementation of coverage path planner on fishing harbour case study

The developed algorithm applied to Chennai's Kasimedu fishing harbour, as shown in Fig. 30(a) and the coordinates of the map are obtained from Geojson.io (2021). Also, it is already discussed in Section 3.1 that the numbers of nodes in the tree grow abruptly with the number of concave vertices in the initial input polygon. So, it is always better to divide a large problem into a set of sub-problem. In Kasimedu fishing harbour, the map is strategically divided into seven sub-regions depending on the port's berth region and is shown in Fig. 30(b). The developed CPP algorithm is implemented on these sub-regions separately for two extreme span limiting condition zero and infinity, and then optimised considering all sub-regions for reducing the transition length by the developed ACO based trajectory optimiser. The result of implementation for span limiting condition zero and infinity on 10 m grid size is shown in Figs. 30(c–d). In Figs. 30(c–d), ACO recommends the intended operation starting from sector one polygon and shifts to subsequent sector polygon in ascending order until all the polygons are travelled by the vehicle. ACO parameters N_{ants}, α, β, γ, ρ, ζ and iteration for output shown in Figs. 30(c–d) are 256, 10, 10, 1, 0.5, 0.5 and 10. The CPP trajectory length summation of each polygon for the span limiting zero and infinity cases is shown in Figs. 30(c–d) are 46752.8 and 47024.5 m. The total Euclidean polygon transition distance for the span limiting zero and infinity cases is shown in Figs. 31(c–d) are 979.7 and 1012.3 m. The total number of turns for the span limiting zero and infinity cases is shown in Figs. 30(c–d) are 353 and 405. The execution time for the simulation is shown in Figs. 30 (c–d) on the personal computer having AMD company's Ryzen 5 3500U, 8 GB RAM, 64-bit and Windows 10 operating system are 176.34 and 188.47 s.

4.2. Analysis of proposed algorithm parameters

The proposed algorithm's performance is influenced by many parameters, which are analysed in detail in this Section. The parameters of the tree search model for unification and decomposition operation are analysed initially in this Section, followed by the analyses of ACO parameters.

The span limiting condition σ considered in the unification operation tree search model is the significant parameter that influences the performance of the proposed method. As discussed in Section 3.2, the span limiting condition decides the feasible operation in the unification tree, and this condition significantly affects the number of turns and polygons count obtained from the proposed algorithm. The number of turns obtained from Chennai's Kasimedu fishing harbour case study for various span limiting conditions is given in Table 1.

It can be inferred from Table 1 that the number of turns is increased with an increase in span limiting conditions. The coverage path plan obtained for the fishing harbour case study for span limiting conditions 50 and 100 are shown in Figs. 31(a–b). The CPP obtained for the span limiting conditions 150 and ∞ are same, which implies that any value for span limiting condition more than 150 does not produce an output polygon having a common visible region for implementing the BF pattern. It is also observed from Figs. 30(c–d) and Figs. 31(a–b), the polygon count reduces as the span limiting condition increases because of feasibility condition for polygon unification operation relaxed as discussed in Section 3.2. The other parameter that affects the performance of the proposed algorithm is grid size. An increase in grid size requires more time for computation, but the vehicle's payload sensors define the grid size. So, studying the proper grid size is out of scope in this work, and the proposed algorithm is developed to produce CPP for any grid size.

Apart from the span limiting condition, other parameters which predominantly affects the proposed method are ACO parameters. The significant parameters of ACO which effects the proposed algorithm are N_{ants}, α, β, ρ and ζ. To study the behaviour of these parameters,

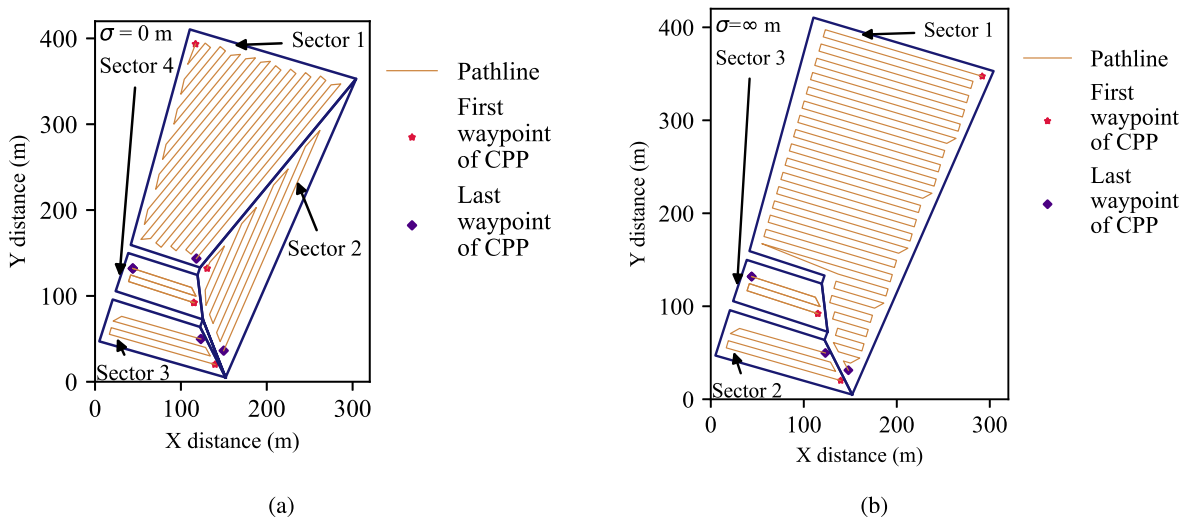


Fig. 29. Graph showing the CPP generated on each polygon by ACO for the polygon set obtained after recursive decomposition and unification operation for span limiting conditions (a) 0 and (b) ∞ m considering the grid size of 8 m.

Table 1
Number of turns obtained for Chennai's Kasimedu fishing harbour case study for various span limiting conditions.

Span limiting condition σ (m)	Number of turns	N_{ants}	α	β	ρ	ζ	$N_{iteration}$	Grid size (m)	Overall Euclidean polygon transition distance (m)	Overall path distance (m)	Time taken for simulation (s)
0	353	256	10	10	0.5	0.5	10	10	979.65	46752.76	176.34
50	360	256	10	10	0.5	0.5	10	10	1066.71	46725.65	179.42
100	370	256	10	10	0.5	0.5	10	10	1061.68	46779.35	183.55
150	405	256	10	10	0.5	0.5	10	10	1012.27	47024.54	188.47
∞	405	256	10	10	0.5	0.5	10	10	1012.27	47024.54	188.47

the case study example whose span limiting condition ∞ is taken for investigation. For understanding N_{ants} effect on the output of trajectory length optimiser as discussed in Section 3.4, multiple simulations for the taken case study are run with different N_{ants} such as 4, 16, 64 and 256, and overall Euclidean transition distances are observed. In these simulations, other parameters such as α, β, ρ and ζ are maintained at 1, 1,0.1 and 1. The result of this simulation is shown in Figs. 32(a–b), and it is observed from these simulation increase in N_{ants} ensures convergence to least overall Euclidean transition distance value fastly.

To study effect of pheromone weightage parameter α, multiple simulations are run with different α such as 0.1, 1 and 10 while other parameters such as N_{ants}, β, ρ and ζ are maintained at 16, 1, 0.1 and 1. The result of these simulations is shown in Figs. 33(a–b). It is observed from these simulations initially overall Euclidean distance for α whose value ten is not performing well. But after a few iterations, it converges better, which is due to subsequent pheromone deposition at every iteration becoming stronger and driving the overall Euclidean distance to the least value.

To investigate the effect of the transition distance weightage parameter β, multiple simulations are run with different β such as 0.1,1 and 10 while other parameters N_{ants}, α, ρ and ζ are maintained at 16, 1, 0.1 and 1. The result of these simulations is shown in Figs. 34(a–b). It is observed from these simulations that an increase in β enables the algorithm to find the least value for overall Euclidean transition distance at the initial stage of iterations itself. From Eq. (28), it can be inferred that higher β ensures more probability for the path with the least Euclidean transition distance, which drives the algorithm to the least value at the initial stage itself.

Furthermore, for analysing the effect of pheromone evaporation rate ρ, multiple simulations are run with different ρ value such as 0.01, 0.1, 0.3 and 0.7 while other parameters such as N_{ants}, α, β and ζ are maintained at 16, 1, 1 and 1. The result of these simulations is shown in Figs. 35(a–b). These simulations show that the increase in ρ ensures convergence to least overall Euclidean transition distance rapidly. On the other hand, the higher ρ erases the previous best path memory on the pheromone matrix τ, so the algorithm converges to some other overall Euclidean distance value instead of an even better least value which is observed in the case of ρ whose value 0.3 and 0.7 in Fig. 35(b).

To study the effect of pheromone deposition rate ζ, multiple simulations are run with different ζ value such as 0.1, 0.5, 0.7 and 1 while other parameters such as N_{ants}, α, β and ρ are maintained at 16, 1, 1 and 0.1. The result of these simulations is shown in Figs. 36(a–b). It is observed from these simulations for this case, convergence rate slightly reduces as ζ decreases, and it is inferred from Fig. 36(a) for ζ whose value is 0.1. Also, the best overall Euclidean transition distance converges to a slightly higher value than the overall best value when the value of ζ reduces, which is inferred from Fig. 36(b). Based on the analyses, it can be recommended that while tuning the algorithm, the first importance must be given to N_{ants} because it is inferred from Figs. 32(a–b), overall Euclidean transition distance converges to least value at a rapid rate when N_{ants} is increased. Then, the importance should be given to α and β parameters because it can be inferred from Figs. 33(a–b) and Figs. 34(a–b) significantly influences the convergence rate and value of convergence, at last, importance given to ρ and ζ parameters.

4.3. Comparison with a published result

In this Section, the developed CPP method is compared against the Exact Cellular Decomposition (ECD) method (Jiao et al., 2010; Li et al., 2011) and Interior Extension of Edges (IEE) (Nielsen et al., 2019) method available in the literature.

The polygon shown in Fig. 37(a) is taken from existing ECD based CPP literature (Jiao et al., 2010). The respective polygon is considered a benchmark to evaluate the proposed CPP algorithm with the existing ECD based CPP algorithm. The ECD base CPP is implemented

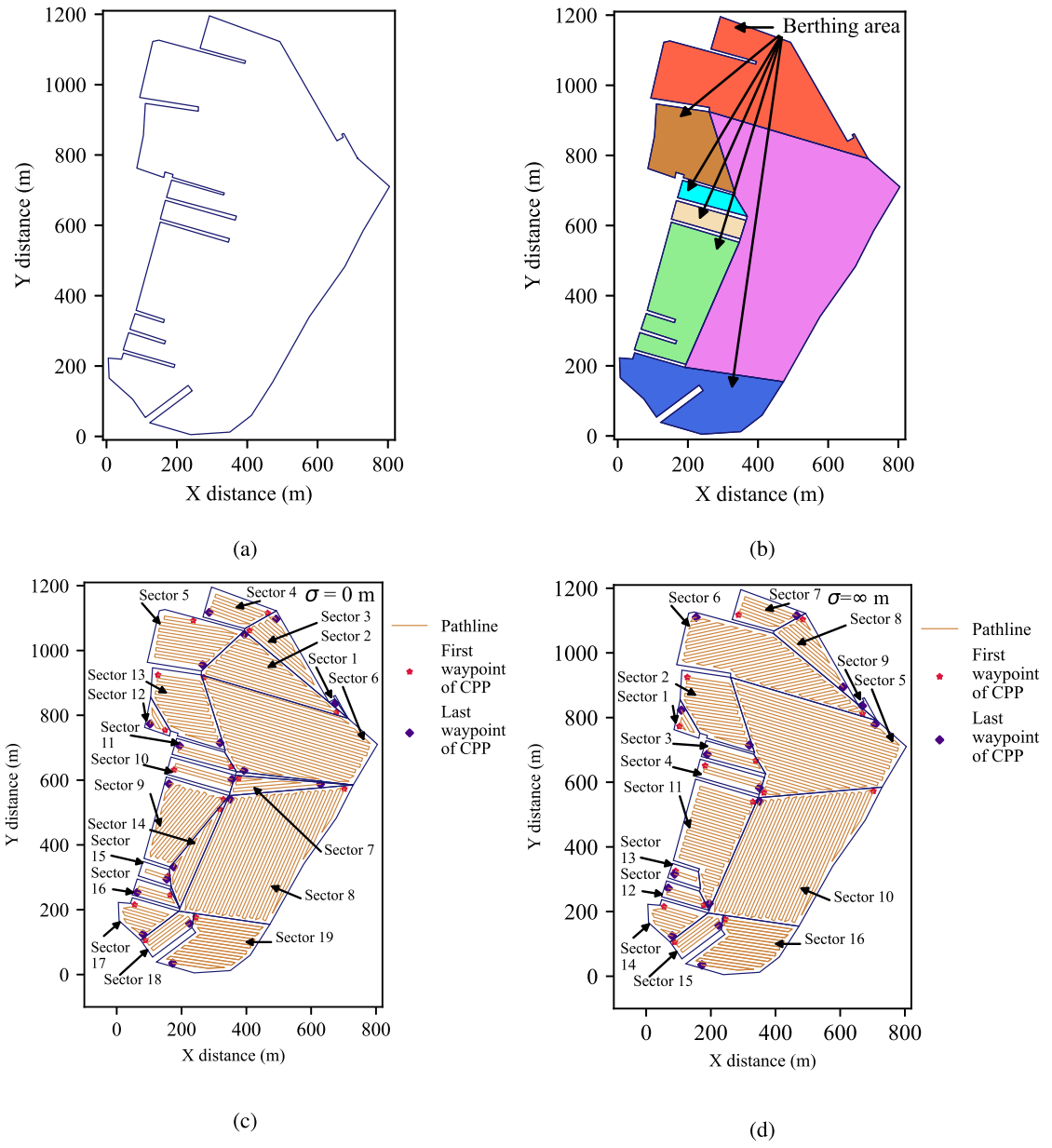


Fig. 30. Graph showing the (a) map of Chennai's Kasimedu fishing harbour, (b) subdivided map for the CPP implementation, and the implemented CPP considering the span limiting conditions (c) 0 and (d) ∞ m for 10 m waypoint grid size.

on the benchmark polygon with a grid size of 0.16 km, and the respective results obtained from the implementation are taken from the literature (Jiao et al., 2010) is shown in Fig. 37(b). The application considered in the literature for the CPP is shown in Fig. 37(b) is the unmanned aerial vehicle. To make a proper comparison, the developed BF based CPP algorithm is implemented on the same benchmark polygon with a grid size of 0.16 km and span limiting condition zero (considered because this condition provides the best least turn). The application for this implementation is the unmanned aerial vehicle, which means filter polygon, as discussed in Section 3.3, is not required. So, the vehicle is allowed to cross the polygon boundaries during the inspection. The proposed CPP method is implemented on the benchmark polygon, and the result is shown in Fig. 37(d). It is concluded from the study that the proposed CPP generated only 82 turns in the path, whereas the existing ECD produced 87 turns in the path for the benchmark polygon. The coverage path plan's overall number of turns is the summation of the number of turns the vehicle undergoes inside each polygon and the number of turns the vehicle undergoes while moving from one to another. The proposed method's subsequent unification operation after decomposition operation reduces the polygon count, which the ECD algorithm did not consider. So the number of turns the vehicle has undergone while moving from one to another polygon lesser for the proposed method, which in turn reduces the overall turns. Also, less polygon provides less major path line direction updation in the vehicle during the inspection. The execution time for the simulation shown in Fig. 37(d) on the personal computer having Advanced Micro Devices (AMD) company's Ryzen 5 3500U, 8 GigaByte (GB) Random Access Memory (RAM), 64-bit and Windows 10 operating system is 11.17 s.

The proposed methods are also compared against the IEE method (Nielsen et al., 2019) based on the polygon's least length span. IEE method is implemented on the same benchmark polygon and the output polygon obtained is taken from the literature (Nielsen et al., 2019) and is shown in Fig. 37(c). IEE method focused on decomposing the input polygon into many convex polygons such that the summation of width or least length span of all convex polygon is least. It is well-known that implementing a back and forth pattern whose sweep line along the least

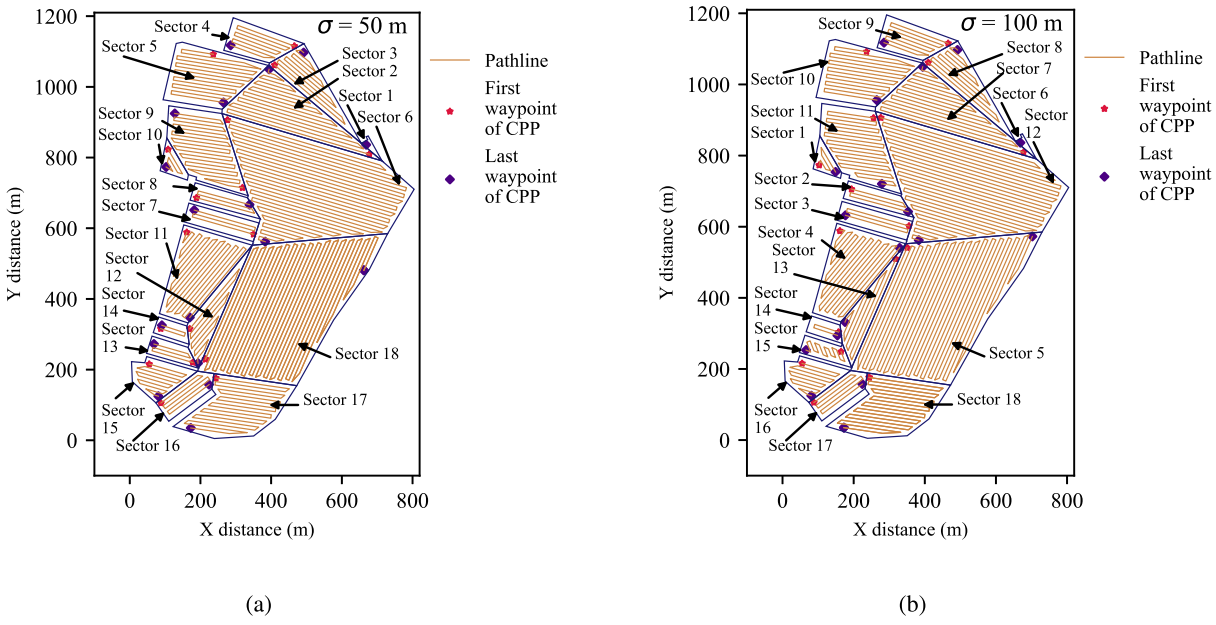


Fig. 31. Graph showing the coverage path plan of Chennai's Kasimedu fishing harbour obtained for the span limiting conditions (c) 50 and (d) 100 m for 10 m waypoint grid size.

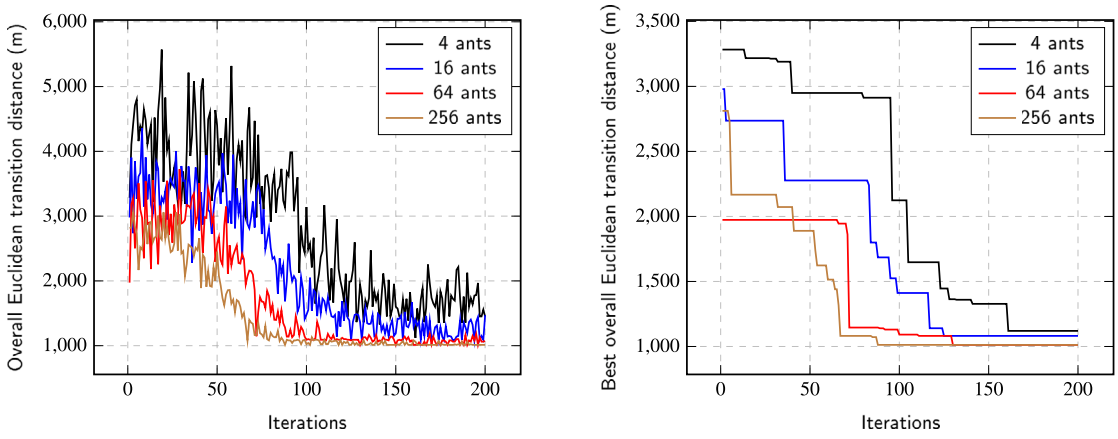


Fig. 32. Graph showing the variation of (a) overall Euclidean transition distance and (b) best overall Euclidean transition distance with respect to iterations for various N_{ants} conditions such as 4, 16, 64 and 256.

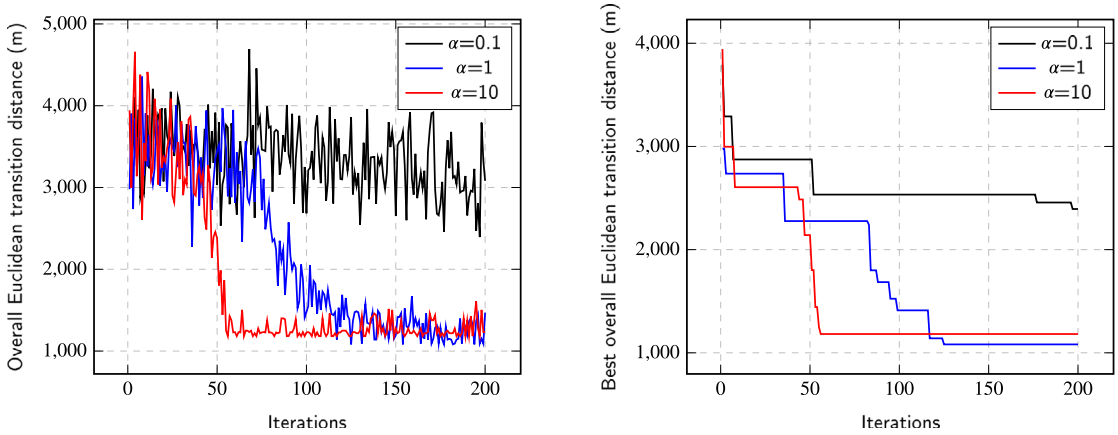


Fig. 33. Graph showing the variation of (a) overall Euclidean transition distance and (b) best overall Euclidean transition distance with respect to iterations for various α conditions such as 0.1, 1 and 10.

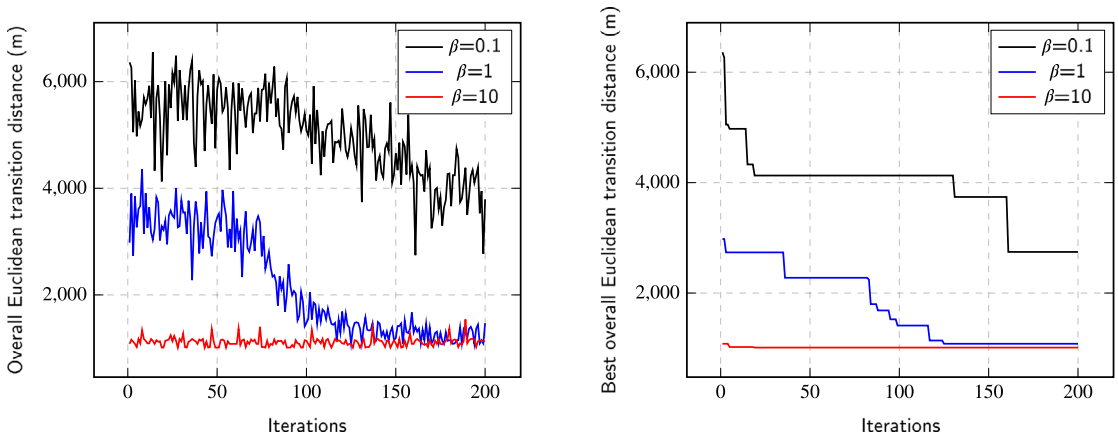


Fig. 34. Graph showing the variation of (a) overall Euclidean transition distance and (b) best overall Euclidean transition distance with respect to iterations for various β conditions such as 0.1, 1 and 10.

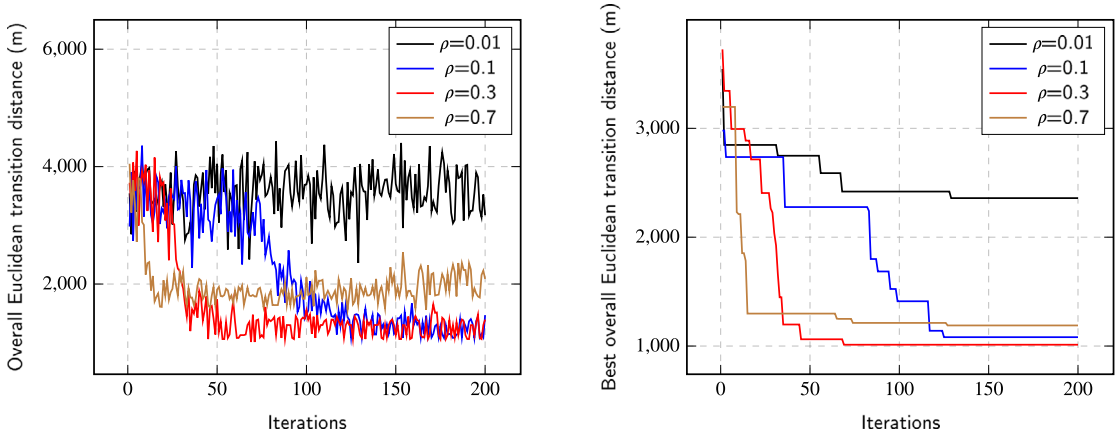


Fig. 35. Graph showing the variation of (a) overall Euclidean transition distance and (b) best overall Euclidean transition distance with respect to iterations for various ρ conditions such as 0.01, 0.1, 0.3 and 0.7.

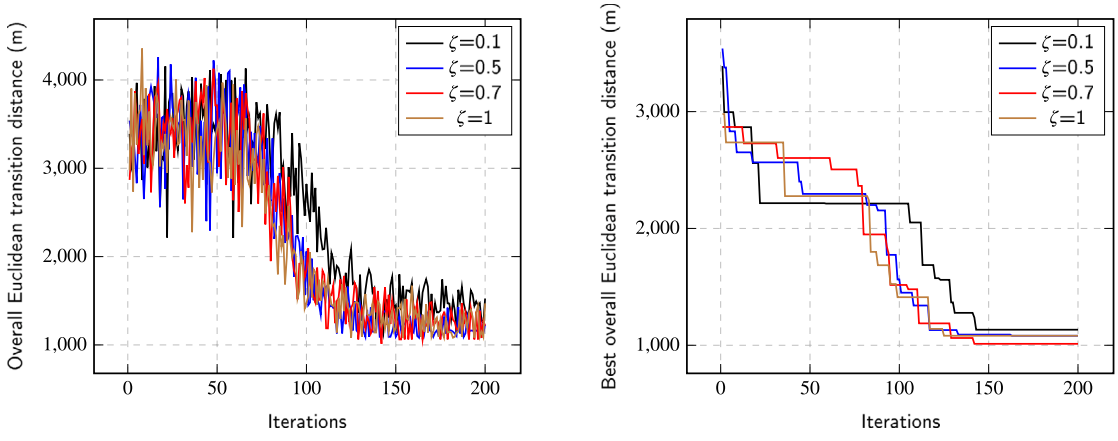


Fig. 36. Graph showing the variation of (a) overall Euclidean transition distance and (b) best overall Euclidean transition distance with respect to iterations for various ζ conditions such as 0.1, 0.5, 0.7 and 1.

length span direction always yields fewer turns. So, the summation of all convex polygon's width produced by the IEE method is compared against the summation of the least viable (for implementing BF pattern) span length of all polygons generated by the proposed method. It is observed that the proposed method produces 4.3% more span length compared to the IEE method but which is marginal. However other aspects been concerned, the proposed method produces fewer polygon counts than IEE method because of unification operation considered in the proposed method. As similar to the ECD method, the IEE method also produces more turns while shifting from one to another polygon in which the proposed algorithm in the manuscript performs better.

5. Conclusions

This work proposes the back and forth based coverage path planner for the autonomous vehicle. The major contribution of this work is

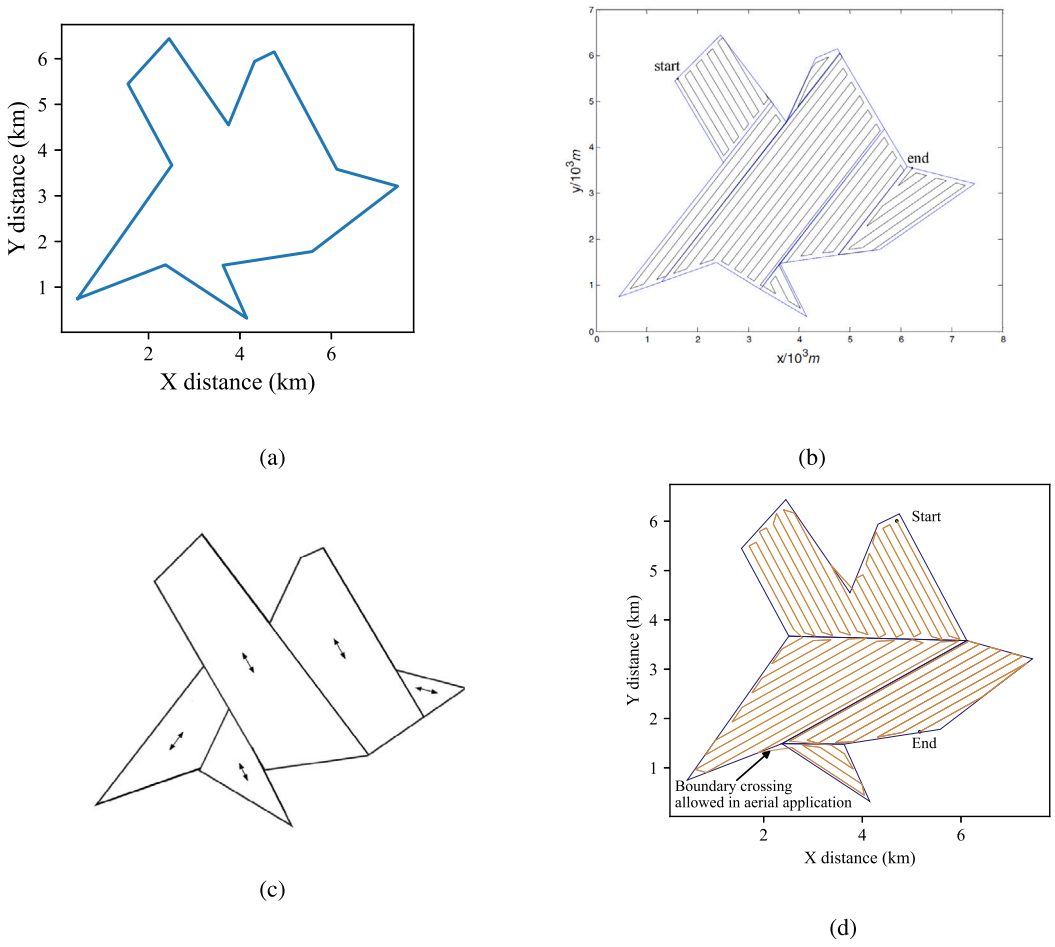


Fig. 37. Graph showing the (a) benchmark polygon from the literature (Jiao et al., 2010) considered for comparative study and the resultant coverage path obtained by implementing (b) the ECD based CPP method (Jiao et al., 2010), (c) IEE based CPP method (Nielsen et al., 2019) and (d) proposed coverage path plan method on the benchmark polygon for the aerial inspection application.

the tree search models for the polygon decomposition and unification operation, which produces the polygons viable for back and forth pattern implementation with the least turns when solved by the Dijkstra algorithm. Another major contribution in this work is developing a trajectory optimiser model equivalent to the travelling salesman problem that produces the least trajectory length when solved by the ant colony optimisation algorithm. As far as future scope is concerned, the proposed algorithm needs to be made for other patterns such as spiral and Hilbert curve patterns. Another prospect the proposed algorithm can be further researched is the heuristic-based algorithm instead of the Dijkstra algorithm for fastly solving the proposed tree search model for polygon decomposition and unification operation. Also, this work utilised the Ant Colony Optimisation for solving trajectory length optimiser model, but this model can also be solved by other algorithms such as Particle Swarm Optimisation and Genetic algorithm, which are further to be researched.

CRediT authorship contribution statement

Karthikeyan Mayilvaganam: Conceptualization, Methodology, Software, Investigation, Writing – Original Draft, Visualization. **Anmol Shrivastava:** Project administration. **Prabhu Rajagopal:** Supervision, Guidance, Funding support, Project management, Writing – Review & Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper

Acknowledgements

This work is supported by the Ministry of Shipping (MoS), India, Government of India (GoI) through the National Technology Centre for Ports, Waterways and Coasts (NTCPWC), IIT Madras, India (Grant number : MEE/18-19/404/MOSH/PRAB). This article does not contain any studies with human participants or animals performed by any of the authors.

References

Ariyasingha, IDID, Fernando, TGI, 2015. Performance analysis of the multi-objective ant colony optimization algorithms for the traveling salesman problem. Swarm Evol. Comput. 23, 11–26.

Artemenko, Oleksandr, Dominic, Omachonu Joshua, Andryeyev, Oleksandr, Mitschele-Thiel, Andreas, 2016. Energy-aware trajectory planning for the localization of mobile devices using an unmanned aerial vehicle. In: 2016 25th International Conference on Computer Communication and Networks. ICCCN, IEEE, pp. 1–9.

Cabreira, Tauã M, Brisolara, Lisane B, Ferreira, Jr., Paulo R, 2019. Survey on coverage path planning with unmanned aerial vehicles. Drones 3 (1), 4.

Cabreira, Taua M, Di Franco, Carmelo, Ferreira, Paulo R, Buttazzo, Giorgio C, 2018. Energy-aware spiral coverage path planning for uav photogrammetric applications. IEEE Robot. Autom. Lett. 3 (4), 3662–3668.

Chai, Runqi, Tsourdos, Antonios, Savvaris, Al, Chai, Senchun, Xia, Yuanqing, 2021. Solving constrained trajectory planning problems using biased particle swarm optimization. IEEE Trans. Aerosp. Electron. Syst..

Choset, Howie, 2000. Coverage of known spaces: The boustrophedon cellular decomposition. Auton. Robots 9 (3), 247–253.

Dogru, Sedat, Marques, Lino, 2015. Energy efficient coverage path planning for autonomous mobile robots on 3D terrain. In: 2015 IEEE International Conference on Autonomous Robot Systems and Competitions. IEEE, pp. 118–123.

Geojson.io, 2021. Retrieved from http://geojson.io/#map=2/20.0/0.0.

Gomez, Juan Irving Vasquez, Melchor, Magdalena Marciano, Lozada, Juan Carlos Herrera, 2017. Optimal coverage path planning based on the rotating calipers algorithm. In: 2017 International Conference on Mechatronics, Electronics and Automotive Engineering. ICMEAE, IEEE, pp. 140–144.

Huang, Wesley H., 2001. Optimal line-sweep-based decompositions for coverage algorithms. In: Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation. Vol. 1. Cat. No. 01CH37164, IEEE, pp. 27–32.

Jiang, Yanqing, Li, Ye, Su, Yumin, Zhou, Ziye, Ma, Teng, An, Li, He, Jiayu, 2018. Route optimizing and following for autonomous underwater vehicle ladder surveys. Int. J. Adv. Robot. Syst. 15 (6), 1729881418813271.

Jiao, Yu-Song, Wang, Xin-Min, Chen, Hai, Li, Yan, 2010. Research on the coverage path planning of uavs for polygon areas. In: 2010 5th IEEE Conference on Industrial Electronics and Applications. IEEE, pp. 1467–1472.

LaValle, Steven M., 2006. Planning Algorithms. Cambridge University Press.

Li, Yan, Chen, Hai, Er, Meng Joo, Wang, Xinmin, 2011. Coverage path planning for UAVs based on enhanced exact cellular decomposition method. Mechatronics 21 (5), 876–885.

Li, Bifan, Wang, Lipo, Song, Wu, 2008. Ant colony optimization for the traveling salesman problem based on ants with memory. In: 2008 Fourth International Conference on Natural Computation. Vol. 7. IEEE, pp. 496–501.

Matplotlib, 2007. Module in Python Language. Retrieved from https://matplotlib.org/stable/index.html.

Nielsen, Lasse Damtoft, Sung, Inkyung, Nielsen, Peter, 2019. Convex decomposition for a coverage path planning for autonomous vehicles: Interior extension of edges. Sensors 19 (19), 4165.

Pirzadeh, Hormoz, 1999. Computational geometry with the rotating calipers.

Python Software Foundation, 2021. Python Language Reference, version 3.8. Retrieved from https://www.python.org/.

Shamos, Michael Ian, 1975. Geometric complexity. In: Proceedings of the Seventh Annual ACM Symposium on Theory of Computing. pp. 224–233.

Shi, Jianguang, Zhou, Mingxi, 2020. A data-driven intermittent online coverage path planning method for AUV-based bathymetric mapping. Appl. Sci. 10 (19), 6688.

Torres, Marina, Pelta, David A, Verdegay, José L, Torres, Juan C, 2016. Coverage path planning with unmanned aerial vehicles for 3D terrain reconstruction. Expert Syst. Appl. 55, 441–451.

Tsourdos, Antonios, White, Brian, Shanmugavel, Madhavan, 2010. Cooperative Path Planning of Unmanned Aerial Vehicles. Vol. 32. John Wiley & Sons.

Yu, Jinglun, Su, Yuancheng, Liao, Yifan, 2020. The path planning of mobile robot by neural networks and hierarchical reinforcement learning. Front. Neurorobotics 14.

Zhu, Daqi, Chen, Tian, Sun, Bing, Luo, Chaomin, 2019. Complete coverage path planning of autonomous underwater vehicle based on GBNN algorithm. J. Intell. Robot. Syst. 94 (1), 237–249.